

Hardware Implementation and Performance Analysis of Sparse LMS Adaptive Filtering Using ZA LMS and RZA LMS Algorithms

M.TECH PROJECT – II
(EC57004) report submitted to

Indian Institute of Technology Kharagpur
In partial fulfillment for the award of the degree of

**Bachelor of Technology & Master of Technology (Dual Degree) in
Electronics and Electrical Communication Engineering with Specialization in
“Microelectronics and VLSI Circuits”**

by

**Gangavaram Srinivasa Teja
21EC37028**

Under the supervision of

Dr. Mrityunjay Chakraborty



DEPARTMENT OF Electronics and Electrical Communication Engineering

INDIAN INSTITUTE OF TECHNOLOGY, KHARAGPUR

KHARAGPUR-721302, INDIA

2025-2026

**DEPARTMENT OF ELECTRONICS AND ELECTRICAL
COMMUNICATION ENGINEERING
INDIAN INSTITUTE OF TECHNOLOGY KHARAGPUR**



CERTIFICATE

This is to certify that the project report entitled “**Hardware Implementation and Performance Analysis of Sparse LMS Adaptive Filtering Using ZA LMS and RZA LMS Algorithms**” submitted by Gangavaram Srinivasa Teja (Roll No. 21EC37028) to Indian Institute of Technology Kharagpur towards partial fulfilment of requirements for the award of degree of Bachelor of Technology & Master of Technology (Dual Degree) in Electronics and Electrical Communication Engineering with Specialization in “**Microelectronics and VLSI Circuits**” is a record of bonafide work carried out by him under my supervision and guidance during spring Semester, 2025-26.

Professor Mrityunjoy Chakraborty

Date: April 27, 2026

Place: Kharagpur

Department of Electronics and Electrical
Communication Engineering
Indian Institute of Technology Kharagpur

DECLARATION

I certify that:

- a) The work contained in this report is original and has been done by me under the guidance of my supervisor(s).
- b) The work has not been submitted to any other Institute for any degree or diploma.
- c) I have followed the guidelines provided by the Institute in preparing the report.
- d) I have conformed to the norms and guidelines given in the Ethical Code of Conduct of the Institute.
- e) Wherever I have used materials (data, theoretical analysis, figures, and text) from other sources, I have given due credit to them by citing them in the text of the report and giving their details in the references. Further, I have taken permission from the copyright owners of the sources, wherever necessary.

Signature of the Student

Gangavaram Srinivasa Teja

21EC37028

Date: 27 April, 2026

Place: Kharagpur

ACKNOWLEDGMENT

I express my sincere gratitude to **Prof. Mrityunjoy Chakraborty**, Department of Electronics and Electrical Communication Engineering, Indian Institute of Technology Kharagpur, for his constant guidance, technical insight, and academic supervision throughout the duration of this work. His expert suggestions and constructive feedback were instrumental in shaping the direction and quality of this project. I am truly grateful for the opportunity to work under his mentorship.

Finally, I am deeply thankful to my family and friends for their constant motivation, moral support, and understanding throughout this journey. Their encouragement has been a source of strength and helped me stay focused throughout the completion of this work.

ABSTRACT

Adaptive filters play a fundamental role in signal processing applications such as system identification, channel equalization, and noise cancellation. The Least Mean Squares (LMS) algorithm, owing to its simplicity and low computational complexity, remains the most widely used adaptive filtering algorithm in both software and hardware implementations. However, its direct hardware realization suffers from a feedback-induced critical path that limits the achievable clock frequency, and its convergence performance degrades significantly when applied to sparse systems where most of the unknown system's coefficients are zero or near-zero.

This work presents the hardware implementation and performance analysis of four adaptive filter variants — standard LMS, Delayed LMS (DLMS), Zero-Attracting LMS (ZA-LMS), and Reweighted Zero-Attracting LMS (RZA-LMS) — targeting sparse system identification on an FPGA platform. All implementations are developed as 16-tap FIR-based adaptive filters in synthesizable Verilog HDL using Q4.12 signed fixed-point arithmetic, and evaluated through simulation and synthesis in the Xilinx Vivado design suite.

The DLMS algorithm addresses the hardware bottleneck by introducing a pipeline depth of $L = 3$ through retiming, breaking the critical feedback path and achieving an improvement in maximum clock frequency over the baseline LMS design. The ZA-LMS algorithm incorporates an L1 norm penalty into the weight update through a dedicated zero attractor unit, enabling sparsity-aware adaptation that accelerates convergence and reduces steady-state mean square error on sparse channels compared to standard LMS. The RZA-LMS algorithm further refines this by applying a reweighted attraction inversely proportional to each coefficient's magnitude, achieving more selective sparsity exploitation with reduced bias on dominant coefficients, at the cost of increased hardware complexity due to fixed-point reciprocal approximation.

Simulation results demonstrate the convergence advantages of the sparsity-aware algorithms over standard LMS on sparse channels, and synthesis reports confirm the timing improvement achieved through pipelining in DLMS. A unified comparison of all four implementations in terms of convergence speed, steady-state MSE, critical path delay, and FPGA resource utilization is presented, providing a comprehensive view of the speed-accuracy-complexity trade-offs inherent in hardware adaptive filtering for sparse system identification

Contents

DECLARATION.....	4
ACKNOWLEDGMENT	5
ABSTRACT	6
LIST OF TABLES	10
LIST OF FIGURES	11
CHAPTER 1	12
INTRODUCTION.....	12
1.1Background and Motivation	12
1.2Overview of Adaptive Filtering	13
1.3Limitations of Standard LMS in Hardware	14
1.4Problem Statement	15
CHAPTER 2	16
LITERATURE REVIEW	16
2.1 Adaptive Filtering: Foundational Works	16
2.2 Hardware Implementations of LMS Filters	16
2.3 Pipelined and Delayed LMS Architectures	17
2.4 Sparsity-Aware Adaptive Filtering: ZA-LMS and RZA-LMS	17
2.5 Summary and Research Gaps	18
2.6 Objectives of the Study	18
2.7 Scope of the Study	19
CHAPTER 3	20
LMS Adaptive Filter - Algorithm and Hardware Implementation	20
3.1 Mathematical Formulation	20
3.2 Convergence Behavior and Step-Size Selection	22
3.3 Fixed-Point Representation and Q4.12 Format	23
3.4 Hardware Architecture and Verilog Implementation	24
3.5 Simulation Results and Synthesis Analysis	25
CHAPTER 4	28

Delayed LMS (DLMS) — Pipelining for Hardware Efficiency	28
4.1 Motivation for Pipelining in Adaptive Filters	28
4.2 DLMS Algorithm: Formulation and Retiming Theory	29
4.2 Effect of Pipeline Depth L on Convergence	30
4.4 Hardware Architecture and Verilog Implementation	31
4.5 Simulation Results and Synthesis Analysis	32
CHAPTER 5	35
ZA-LMS — Zero-Attracting LMS for Sparse Systems.....	35
5.1 Sparse System Identification and Motivation	35
5.2 ZA-LMS Algorithm: Formulation and Intuition.....	36
5.3 Effect of the Zero-Attraction Parameter ρ	36
5.4 Hardware Architecture and Verilog Implementation	37
5.5 Simulation Results and Synthesis Analysis	38
CHAPTER 6	42
RZA-LMS — Reweighted Zero-Attracting LMS	42
6.1 Sparse Adaptive Filtering and Limitations of ZA-LMS	42
6.2 RZA-LMS Algorithm: Formulation and Intuition	43
6.3 Effect of Parameters ρ and ϵ on Convergence	43
6.4 Hardware Architecture and Verilog Implementation	44
6.5 Simulation Results and Synthesis Analysis	45
CHAPTER 7	49
Comparative Results and Analysis	49
7.1 Experimental Setup and Parameters	49
7.2 Convergence Comparison: LMS, DLMS, ZA-LMS, RZA-LMS	50
7.3 Synthesis Comparison: Timing and Resource Utilization	51
7.4 Summary and Discussion	53
CHAPTER 8	55
CONCLUSIONS AND FUTURE SCOPE	55
8.1 Summary of Contributions	55

8.2 Limitations of the Current Work	56
8.3 Directions for Future Work	56
REFERENCES	58

LIST OF TABLES

Table 1. Comparison of Fixed-Point Formats Used in LMS Implementation —	21
Table 2. Synthesis Results for LMS Adaptive Filter —	26
Table 3. Synthesis Results Comparison: Retimed DLMS and LMS —	33
Table 4. Synthesis Results for ZA-LMS Adaptive Filter —	39
Table 5. Synthesis Results for Pipelined ZA-LMS (ZA-LMSpip) —	40
Table 6. Synthesis Results for RZA-LMS Adaptive Filter —	46
Table 7. Synthesis Results for Pipelined RZA-LMS (RZA-LMSpip) —	47
Table 8. Simulation and Hardware Parameters Used Across All Implementations —	48
Table 9. Unified Synthesis Results Comparison Across All Implementations —	50

LIST OF FIGURES

- Figure 1. Block Diagram of LMS Adaptive Filter for System Identification — 19
- Figure 2. Hardware Architecture of LMS Adaptive Filter — 23
- Figure 3. Desired Signal $d(n)$ and Filter Output $y(n)$ for LMS — 24
- Figure 4. Absolute Error $|e(n)|$ vs. Iteration for LMS — 25
- Figure 5. Convergence of Filter Weights for LMS — 25
- Figure 6. Block Diagram of DLMS Adaptive Filter — 27
- Figure 7. Hardware Architecture of DLMS Adaptive Filter — 30
- Figure 8. Retimed DLMS Architecture — 31
- Figure 9. Desired Signal $d(n)$ and Filter Output $y(n)$ for Retimed DLMS — 32
- Figure 10. Absolute Error $|e(n)|$ vs. Iteration for Retimed DLMS — 32
- Figure 11. Convergence of Filter Weights for Retimed DLMS — 33
- Figure 12. Hardware Architecture of ZA-LMS Adaptive Filter — 37
- Figure 13. Desired Signal $d(n)$ and Filter Output $y(n)$ for ZA-LMS — 38
- Figure 14. Absolute Error $|e(n)|$ vs. Iteration for ZA-LMS — 38
- Figure 15. Convergence of Filter Weights for ZA-LMS — 39
- Figure 16. Hardware Architecture of RZA-LMS Adaptive Filter — 43
- Figure 17. Desired Signal $d(n)$ and Filter Output $y(n)$ for RZA-LMS — 45
- Figure 18. Absolute Error $|e(n)|$ vs. Iteration for RZA-LMS — 45
- Figure 19. Convergence of Filter Weights for RZA-LMS — 46
- Figure 20. MSE vs. Iteration Comparison Across All Algorithms — 49
- Figure 21. Maximum Clock Frequency (F_{max}) Comparison Across All Implementations — 51
- Figure 22. LUT and Flip-Flop Utilization Comparison Across All Implementations — 52

CHAPTER 1

INTRODUCTION

1.1 Background and Motivation

Signal processing systems operating in real-world environments must continually adapt to changing conditions. Whether the task is noise cancellation in a communication channel, echo suppression in a telephone network, or system identification in control systems, the underlying challenge is the same: the system must learn an unknown relationship between its input and output, and update its behaviour as that relationship evolves over time. This requirement gives rise to the field of **adaptive filtering**, where filter coefficients are not fixed at design time but are iteratively updated based on observed data.

The **Least Mean Squares (LMS)** algorithm, introduced by Widrow and Hoff in 1960, remains one of the most widely used adaptive algorithms in both software and hardware implementations. Its appeal lies in its simplicity — the weight update requires only a multiplication, an addition, and access to recent input samples. Despite this simplicity, LMS achieves robust convergence under stationary conditions and requires no knowledge of the input signal statistics.

However, as digital signal processing has migrated from general-purpose processors to dedicated hardware FPGAs, **ASICs**, and embedded DSP cores new constraints have emerged. Hardware implementations demand not just algorithmic correctness but also **timing closure**, **area efficiency**, and **throughput maximization**. The sequential nature of the LMS weight update introduces a **feedback loop** that limits the achievable clock frequency, making direct hardware implementation of standard LMS a bottleneck in high-speed applications.

Simultaneously, many real-world systems such as sparse multipath communication channels, acoustic echo paths, and network channel models exhibit **sparsity**: most of the system's impulse response coefficients are zero or near-zero, with only a few dominant taps. Standard LMS treats all coefficients equally and fails to exploit this structure, leading to suboptimal convergence. These two gaps hardware inefficiency and sparsity blindness form the central motivation for this work.

1.2 Overview of Adaptive Filtering

The An **adaptive filter** is a self-adjusting system that modifies its parameters based on an error signal derived from the difference between a desired output and its own filtered output. The general structure consists of two components: a **digital filter** typically a **Finite Impulse Response (FIR)** filter and an **adaptive algorithm** that updates the filter's coefficients in response to the error.

The FIR filter output at discrete time step n is computed as:

$$y(n) = w^T(n)x(n)$$

where $w(n)$ is the weight vector and $x(n)$ is the input vector of length N The

error signal is defined as:

$$e(n) = d(n) - y(n)$$

where $w(n)$ is the weight vector and $x(n)$ is the input vector of length N

The LMS algorithm updates the weight vector using the **steepest descent** principle with an instantaneous gradient estimate:

$$w(n+1) = w(n) + \mu \cdot e(n) \cdot x(n)$$

Here, μ is the **step size** (or learning rate), which controls the trade-off between convergence speed and steady-state error. A larger μ accelerates convergence but increases the **excess mean square error (EMSE)** at steady state; a smaller μ yields lower EMSE but slower adaptation.

The principal advantage of LMS over other adaptive algorithms such as **Recursive Least Squares (RLS)** is its low computational complexity of $O(N)$ per iteration, where N is the filter order. RLS achieves faster convergence but at $O(N^2)$ cost, making it impractical for hardware implementation at large filter orders. LMS strikes a favorable balance, which explains its enduring dominance in hardware-oriented adaptive filtering applications.

Adaptive filters find application in **channel equalization**, **system identification**, **beamforming**, **acoustic echo cancellation**, and **noise cancellation**. In all these contexts, the filter must converge

to the unknown system's impulse response and track it if it changes a requirement that places direct demands on both algorithmic stability and hardware throughput.

1.3 Limitations of Standard LMS in Hardware

While the LMS algorithm is computationally simple in software, its direct hardware implementation exposes a fundamental structural problem: a **loop-carried data dependency** in the weight update path.

At each clock cycle, the hardware must: compute the filter output $y(n)$, compute the error $e(n)$, multiply the error by each input sample to get the gradient, and finally update each weight $w(n+1)$ before the next input sample arrives. Each of these steps depends on the result of the previous one. This forms a **critical feedback loop** whose propagation delay directly limits the maximum achievable **clock frequency** ($F_{\max}$).

In an N -tap LMS filter implemented in hardware, the **critical path** passes through a multiplier (for the inner product), an adder tree (for accumulation), another multiplier (for gradient scaling), and an adder (for weight update). On an FPGA fabric, each of these operations introduces combinational delay. The synthesis tool must close timing within a single clock period, and the presence of this long combinational chain forces either a **reduced clock frequency** or the introduction of pipeline registers — which, if inserted naively, break the algorithm's correctness.

Furthermore, **fixed-point arithmetic** introduces additional challenges. Unlike floating-point, fixed-point representations require careful management of **bit-width**, **overflow**, and **truncation error**. In the Q4.12 format used in this work, all arithmetic including multiplications that produce 32-bit intermediate products must be truncated back to 16 bits with appropriate right shifts. Poor bit-width choices lead to either **coefficient overflow** (causing divergence) or **quantization noise** (degrading steady-state MSE).

These limitations the feedback-induced timing bottleneck and the precision constraints of fixedpoint arithmetic make the direct implementation of LMS inadequate for high-throughput hardware systems, and motivate the exploration of **pipelined variants** such as the **Delayed LMS (DLMS)** algorithm.

1.4 Problem Statement

The central problem addressed in this work lies at the intersection of two axes: **hardware efficiency** and **algorithmic optimality for sparse systems**.

From the hardware perspective, the standard LMS algorithm cannot be freely pipelined due to its inherent weight-feedback dependency. Inserting pipeline stages into the critical path stales the weight vector used for gradient computation, altering the algorithm's behavior. While the **Delayed LMS (DLMS)** formulation resolves this by explicitly accounting for the delay in its update equation relying on retiming theory to justify algorithmic validity its effect on convergence performance must be carefully characterized, particularly as the pipeline depth L increases.

From the algorithmic perspective, standard LMS and DLMS are designed for **dense systems**, where all filter taps are assumed to carry significant energy. In practice, many physical channels including **sparse multipath channels** encountered in wireless communications and underwater acoustics have impulse responses where the majority of taps are zero or negligibly small. Applying a dense adaptive algorithm to such a channel wastes the algorithm's degrees of freedom on nearzero coefficients, leading to **slower convergence** and **higher steady-state error** compared to what is theoretically achievable.

Sparsity-aware algorithms such as **Zero-Attracting LMS (ZA-LMS)** and **Reweighted ZeroAttracting LMS (RZA-LMS)** address this by incorporating a penalty term into the weight update that actively drives small coefficients toward zero. However, translating this additional update term into synthesizable, fixed-point Verilog hardware introduces new design questions around precision, resource cost, and timing.

This work therefore poses the following problem: Can the LMS adaptive filter be restructured for hardware efficiency through pipelining (DLMS), and simultaneously enhanced for sparse channel performance (ZA-LMS, RZA-LMS), while maintaining correctness and synthesizability in a fixed-point FPGA implementation?

CHAPTER 2

LITERATURE REVIEW

2.1 Adaptive Filtering: Foundational Works

The field of adaptive filtering was established by Widrow and Hoff in 1960 with the introduction of the **Least Mean Squares (LMS)** algorithm. Their key insight was that a filter's coefficients could be updated iteratively using a **stochastic gradient descent** rule, requiring no prior knowledge of input signal statistics - replacing expensive matrix-based solutions with a lightweight, sample-by-sample update.

Subsequent decades saw the development of important variants. The **Recursive Least Squares (RLS)** algorithm achieved faster convergence by maintaining an estimate of the inverse input covariance matrix, but at $O(N^2)$ computational cost - impractical for hardware at large filter lengths. The **Normalized LMS (NLMS)** algorithm offered a middle ground, improving robustness to input power variations by normalizing the step size with respect to instantaneous input energy. Theoretical analysis of **convergence, misadjustment, and tracking behavior** under both stationary and non-stationary conditions established the performance boundaries within which all subsequent hardware-oriented variants operate.

2.2 Hardware Implementations of LMS Filters

Early hardware implementations of LMS targeted **ASICs**, where the regularity of the multiplyaccumulate structure mapped naturally onto **systolic array architectures**. With the rise of **FPGAs**, implementation studies diversified, reporting resource usage in terms of **LUTs, flip-flops, and DSP blocks** across various filter orders and clock targets.

A consistent finding across these works is the **weight feedback bottleneck** - the weight update for cycle n must complete before the next sample is processed, creating a long combinational path that limits **Fmax**. Strategies such as **distributed arithmetic** and **block LMS** were proposed, but each introduced trade-offs in area or latency. The impact of **fixed-point word length** and **coefficient quantization** on steady-state **MSE** was also studied extensively, providing the design guidelines that inform the Q4.12 format used in this work.

2.3 Pipelined and Delayed LMS Architectures

The incompatibility between pipelining and the LMS feedback loop was addressed through **retiming theory**, which allows pipeline registers to be repositioned within a circuit without altering its output behavior. Applied to LMS, retiming inserts pipeline stages into the critical feedback path at the cost of introducing an **L-cycle delay** between error computation and weight update — giving rise to the **Delayed LMS (DLMS)** algorithm.

Researchers have shown that DLMS converges to the same Wiener solution as standard LMS under stationary conditions, provided the step size is chosen within stability bounds that depend on both input statistics and pipeline depth L . Studies characterizing DLMS convergence as a function of L report that degradation is gradual and manageable for small L , while the hardware benefit a proportional reduction in critical path delay and increase in clock frequency is significant.

These findings directly motivate the $L = 8$ pipelined implementation developed in this work.

2.4 Sparsity-Aware Adaptive Filtering: ZA-LMS and RZA-LMS

Many practical channels including **sparse multipath wireless channels**, **underwater acoustic paths**, and **network echo paths** have impulse responses where most coefficients are zero or negligibly small. Standard LMS ignores this structure, leading to slower convergence and higher steady-state error than is theoretically necessary.

Drawing from **compressive sensing** and **L1 norm regularization**, the **Zero-Attracting LMS (ZA-LMS)** algorithm augments the standard update with a term proportional to the **sign** of each weight, applying a constant attractive force toward zero on all coefficients. The **Reweighted ZeroAttracting LMS (RZA-LMS)** refines this by making the attraction **inversely proportional to each weight's magnitude**, ensuring dominant coefficients are left undisturbed while near-zero coefficients are driven to zero more selectively. Studies consistently show RZA-LMS outperforms ZA-LMS on highly sparse systems. However, hardware realization of these algorithms particularly the **fixed-point reciprocal** required by RZA-LMS has received limited attention in the literature.

2.5 Summary and Research Gaps

The literature survey presented in the preceding sections reveals a well-established body of knowledge on adaptive filtering algorithms, hardware LMS architectures, pipelined DLMS

formulations, and sparsity-aware variants. However, a careful examination of existing works exposes several gaps that this project addresses.

First, while the theoretical properties of DLMS are well understood, **end-to-end hardware implementations in modern FPGA environments** specifically targeting Vivado synthesis with fixed-point arithmetic at practical filter lengths are less thoroughly documented. Most published works either analyze DLMS at an architectural level without RTL validation, or report results on older FPGA families where timing characteristics differ significantly from current devices.

Second, the hardware implementation of **ZA-LMS and RZA-LMS in synthesizable RTL** remains an underexplored area. The majority of work on these algorithms treats them as software or MATLAB-level constructs, with analysis focused on algorithmic convergence rather than hardware cost, critical path, or fixed-point precision effects. The design of a dedicated **zero attractor unit** in Verilog, and its integration into a complete pipelined datapath, has not been systematically studied.

Third, a **unified comparative study** that evaluates LMS, DLMS, ZA-LMS, and RZA-LMS under identical hardware conditions same filter length, same number format, same FPGA target, same input scenarios is absent from the existing literature. Such a comparison is essential for practitioners making design decisions in sparse channel equalization applications.

This work directly addresses all three gaps through RTL implementation, simulation, and synthesis of all four algorithm variants on a common 16-tap, Q4.12 fixed-point platform targeting Xilinx Vivado.

2.6 Objectives of the Study

Based on the identified research gaps, this work is guided by the following objectives:

1. To implement a functionally correct and synthesizable 8-tap **LMS adaptive filter** in Verilog using Q4.12 fixed-point arithmetic, and establish its convergence behavior and synthesis characteristics as a baseline for all subsequent comparisons.
2. To design and implement a **pipelined DLMS filter** with pipeline depth $L = 8$, breaking the weight-feedback critical path to improve clock frequency, while analyzing the impact of pipelining on convergence speed and steady-state MSE.

3. To implement the **ZA-LMS algorithm** in Verilog for a 8-tap sparse channel identification task, incorporating a fixed-point zero attractor unit, and to quantify the convergence advantage over standard LMS on sparse systems.
4. To investigate the **RZA-LMS algorithm** in hardware, identify the key implementation challenges introduced by the reweighting term, and present available results and design observations.
5. To perform a **systematic comparative analysis** of all four implementations using simulationbased convergence metrics and Vivado synthesis reports, providing a unified view of the performance, timing, and resource trade-offs across the algorithm family.

2.7 Scope of the Study

This study covers **8-tap FIR-based adaptive filters** implemented in **Q4.12 signed fixed-point arithmetic**, targeting a Xilinx FPGA through Vivado synthesis. Convergence analysis is performed using Python on synthetically generated sparse channel models.

The algorithmic scope is limited to LMS, DLMS ($L = 8$), ZA-LMS, and RZA-LMS. Pipelined extensions of ZA-LMS or RZA-LMS, floating-point implementations, and non-stationary channel tracking are recognized as natural future extensions but fall outside this work.

On the hardware side, the study analyzes **timing (Fmax)**, **LUT utilization**, **flip-flop count**, and **DSP block usage** as reported by Vivado post-synthesis. **Post-implementation (place-and-route) timing** and **power analysis** are not the primary focus, though synthesis slack reports are discussed where relevant. On-board FPGA deployment and power analysis are not included.

CHAPTER 3

LMS Adaptive Filter - Algorithm and Hardware Implementation

3.1 Mathematical Formulation

The **Least Mean Squares (LMS)** algorithm solves the problem of **adaptive system identification**: given an unknown system driven by an input signal $x(n)$, the goal is to adjust the coefficients of an N -tap FIR filter such that its output $y(n)$ converges to the unknown system's output $d(n)$. The algorithm achieves this iteratively, using the error between the two outputs to guide each coefficient update.

The output of the adaptive FIR filter at discrete time n is the inner product of the weight vector and the input signal vector:

$$y(n) = w^T(n) \cdot x(n) = \sum w_k(n)x(n-k), \text{ for } k = 0 \text{ to } N-1$$

where $\mathbf{w}(n) = [w_0(n), w_1(n), \dots, w_{N-1}(n)]^T$ is the weight vector and $\mathbf{x}(n) = [x(n), x(n-1), \dots, x(n-N+1)]^T$ is the input regressor vector of length $N = 8$ in this work.

The **error signal** is defined as the difference between the desired response and the filter output:

$$e(n) = d(n) - y(n)$$

In the standard LMS the cost function $L(n)$ is defined as

$$L(n) = \frac{1}{2} e^2(n)$$

The LMS weight update rule applies a correction proportional to the instantaneous gradient of the squared error with respect to each weight:

$$w(n+1) = w(n) - \mu \frac{\partial L(n)}{\partial w(n)}$$

$$w(n+1) = w(n) + \mu \cdot e(n) \cdot x(n)$$

Here, μ is the **step size**, a positive scalar that governs the rate of adaptation. The gradient term $\mu \cdot e(n) \cdot x(n)$ is an instantaneous, noisy estimate of the true gradient of the mean square error surface. Despite this noise, the update is unbiased in expectation, and the weight vector converges to the **Wiener-Hopf optimal solution** $w^* = R^{-1}P$, where R is the input autocorrelation matrix, $R = E[x(n)x^T(n)]$ and P is the crosscorrelation vector between input and desired signal $P = E[x(n)d(n)]$. If λ_{max} is the maximum eigen value of R matrix then the convergence condition for LMS is

$$0 < \mu < \frac{1}{\lambda_{max}}$$

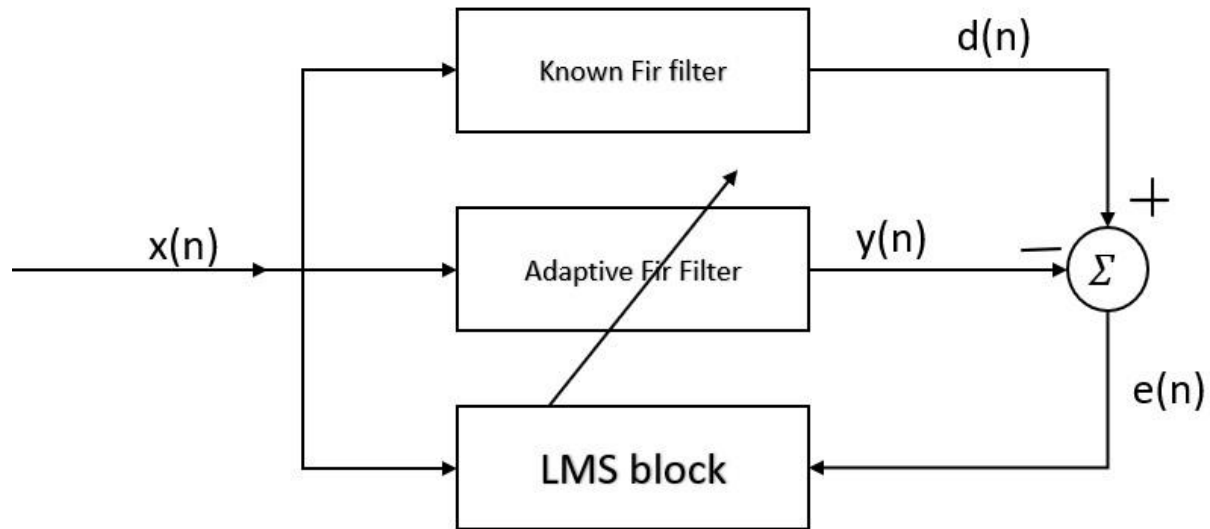


Fig 1. Block Diagram of LMS Adaptive Filter for System Identification

The elegance of this formulation lies in its complete avoidance of matrix operations. Each weight is updated independently using only the scalar error and the corresponding delayed input sample a property that makes LMS inherently parallelizable and hardware-friendly in structure, even if the feedback dependency introduces timing constraints in practice.

3.2 Convergence Behavior and Step-Size Selection

The convergence behavior of the LMS algorithm is governed primarily by the **step size** μ and the **eigenvalue spread** of the input autocorrelation matrix R . For the algorithm to remain stable and converge in the mean square sense, μ must satisfy the stability bound:

$$0 < \mu < 1/\lambda_{\max}$$

where $\lambda_{\max}$ is the largest eigenvalue of R . In practice, since computing $\lambda_{\max}$ is non-trivial, a conservative bound based on the total input signal power is commonly used:

$$0 < \mu < 1 / (N \cdot \sigma_x^2)$$

where σ_x^2 is the variance of the input signal and N is the filter length. For $N = 8$, this bound places a strict upper limit on μ that must be respected in both software simulation and the hardware implementation.

The choice of μ involves a fundamental **speed-accuracy trade-off**. A large μ accelerates initial convergence but results in a higher **steady-state excess mean square error (EMSE)**, as the weight vector continues to fluctuate around the optimal solution rather than settling precisely. A small μ reduces steady-state error but requires significantly more iterations to converge. The **misadjustment** M , defined as the ratio of EMSE to the minimum MSE, approximates to:

Under the independence assumption, the steady state excess MSE is

$$p_{ex}(\infty) = \lim_{n \rightarrow \infty} E[((w(n) - w)^T x(n))^2] = \frac{\eta}{2 - \eta} P_0$$

where P_0 is the power of observation noise

$$P_0 = E[v^2(n)]$$

$$\eta = \text{tr} (R (I - \mu R)^{-1})$$

This expression makes explicit that misadjustment grows with both μ and the filter length N , reinforcing the importance of conservative step-size selection for longer filters.

In the hardware implementation of this work, μ is stored as a fixed-point integer in Q4.12 format. Rather than storing μ directly as a small fractional value, it is represented as a scaled integer for example, a value of $\mu = 0.01$ in real arithmetic corresponds to an integer of approximately 41 in Q4.12 (since $0.01 \times 2^{12} \approx 41$). This representation avoids the need for floating-point logic while preserving sufficient precision for stable convergence.

3.3 Fixed-Point Representation and Q4.12 Format

All arithmetic in the hardware implementation uses **signed fixed-point representation** in the **Q4.12 format**, where each 16-bit word is interpreted as having 1 sign bit, 3 integer bits, and 12 fractional bits. This gives a representable range of approximately -8.0 to $+7.9997$, with a resolution (LSB value) of $2^{-12} \approx 0.000244$.

Format	Total Bits	Integer Bits	Fractional Bits	Representable Range	Resolution (LSB)
Q4.12	16	4	12	-8 to +7.9998	$2^{-12} = 0.00024$
Q8.8	16	8	8	-128 to +127.996	$2^{-8} = 0.003906$
Q2.14	16	2	14	-2 to +1.9999	$2^{-14} = 0.00006$

Table 1. Comparison of Fixed-Point Formats Used in LMS Implementation

The choice of Q4.12 is motivated by two constraints. First, the input signals, desired signals, and filter weights in the system identification scenario are all normalized to lie within $[-1, +1]$, requiring at least a few integer bits as a safety margin against transient overflow. Second, the fractional precision of 12 bits provides sufficient resolution for the weight update to progress meaningfully — a coarser format such as Q8.8 would introduce excessive **quantization noise** that dominates the steady-state MSE.

The most critical arithmetic operation in the LMS datapath is the **weight update multiplication**: computing $\mu \cdot e(n) \cdot x(n-k)$ for each tap k . When two Q4.12 numbers are multiplied, the result is a 32-bit product in Q8.24 format. To return this result to Q4.12 for storage in the weight register, a **12-bit arithmetic right shift** is applied, discarding the lower 12 fractional bits. This truncation introduces a small but systematic **rounding error** that accumulates over iterations and contributes to the steady-state noise floor.

Similarly, the **FIR accumulation** — summing 16 products of Q4.12 numbers — can produce results up to $16\times$ the individual product magnitude, requiring the accumulator to be extended to at least **32 bits** to prevent overflow. After accumulation, the result is right-shifted and truncated back to 16 bits for output. Careful management of these intermediate bit widths is essential: undersized accumulators cause **overflow-driven divergence**, as was encountered and resolved during the debugging phase of this implementation.

3.4 Hardware Architecture and Verilog Implementation

The hardware architecture of the 8-tap LMS adaptive filter is organized into two functionally distinct modules: the **FIR filter core**, which computes the filter output $y(n)$, and the **weight update core**, which computes the gradient and updates all 8 weights. A top-level wrapper instantiates both modules and manages the shared signal interfaces.

The **FIR filter core** maintains a 8-element shift register of signed 16-bit input samples. On each rising clock edge, the input sample is shifted in and the oldest sample is discarded. Each of the 8 taps multiplies the stored sample by the corresponding weight using a **8×8 signed multiplier**, producing a 32-bit product. These products are summed using a **pipelined adder tree** and the result is right-shifted by 12 bits to produce the 8-bit output $y(n)$.

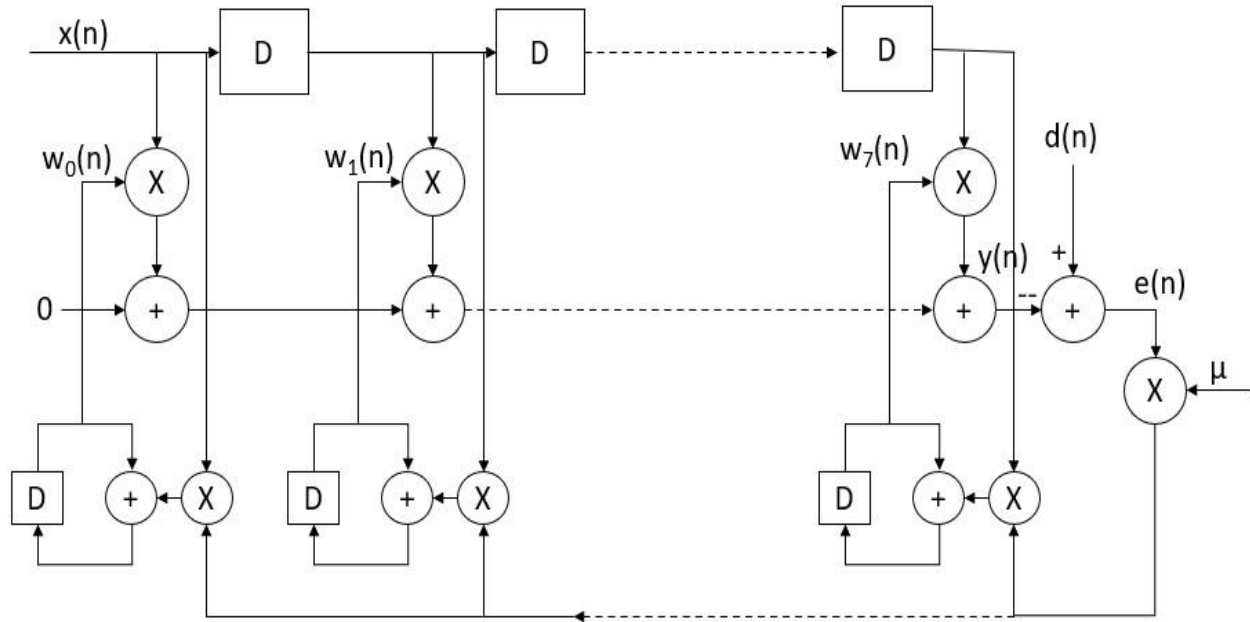


Fig 2. LMS architecture

The **weight update core** receives the error signal $e(n)$ and the 16 delayed input samples. For each tap k , it computes the gradient term as:

$$\Delta w_k(n) = \mu \cdot e(n) \cdot x(n-k)$$

This involves two multiplications: first $e(n) \times x(n-k)$, then scaling by μ . In the Verilog implementation, since μ is stored as an integer (e.g., 41 for $\mu \approx 0.01$ in Q4.12), the scaling is

implemented as a multiply followed by a **24-bit right shift** to account for the combined Q-format scaling. The updated weight is then:

$$w_k(n+1) = w_k(n) + \Delta w_k(n)$$

stored in a 16-bit signed register initialized to zero at reset. All 16 weights are updated **in parallel** on each clock cycle, using **non-blocking assignments** to ensure the current cycle's weights — not the partially updated values — are used consistently throughout the computation.

The critical design consideration in this module is the **feedback dependency**: $y(n)$ depends on $w(n)$, $e(n)$ depends on $y(n)$, and $w(n+1)$ depends on $e(n)$. This chain must complete within a single clock period, making the multiplier-accumulator-multiplier path the **critical path** that determines the maximum achievable clock frequency.

3.5 Simulation Results and Synthesis Analysis

Simulation Setup: The LMS filter was simulated in Vivado using a testbench that models a **sparse system identification** scenario. A 8-tap unknown system with a few dominant coefficients was driven by a white noise input sequence generated in Python and applied to the Verilog testbench via a memory initialization file. The desired signal $d(n)$ is the output of the unknown system, and the adaptive filter is tasked with identifying its impulse response. The step size μ was set to a value within the stability bound derived from the input signal power and filter length.

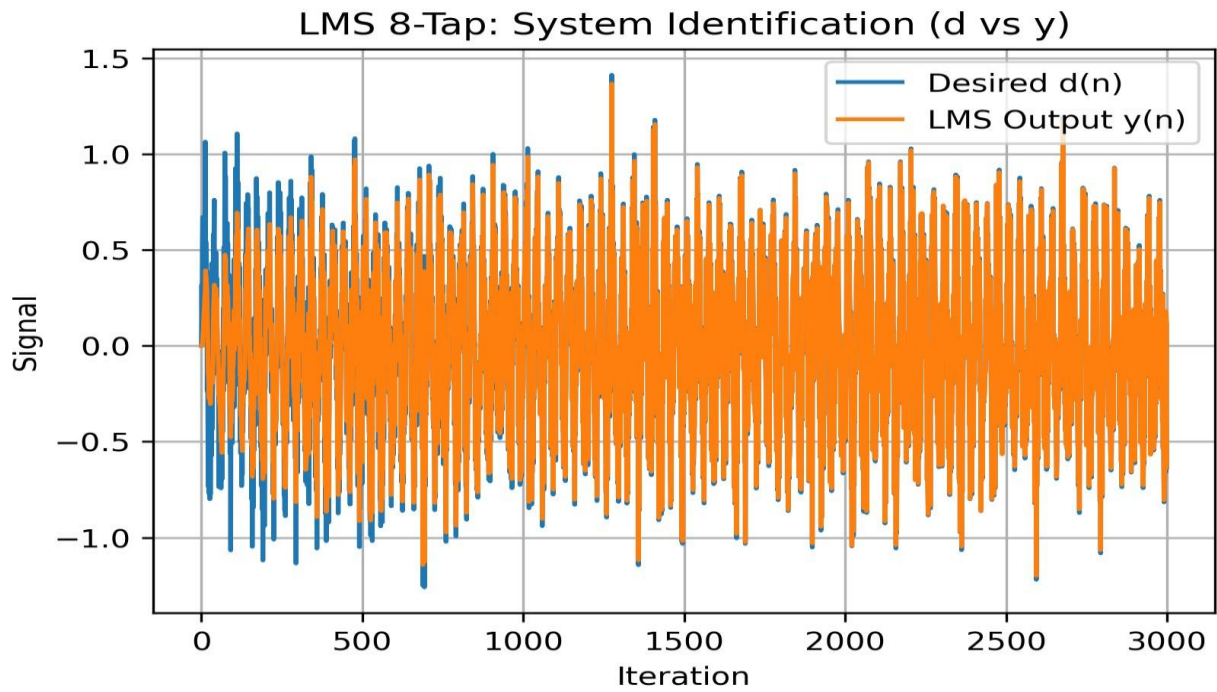


Fig 3. $y(n)$ vs $d(n)$ for LMS

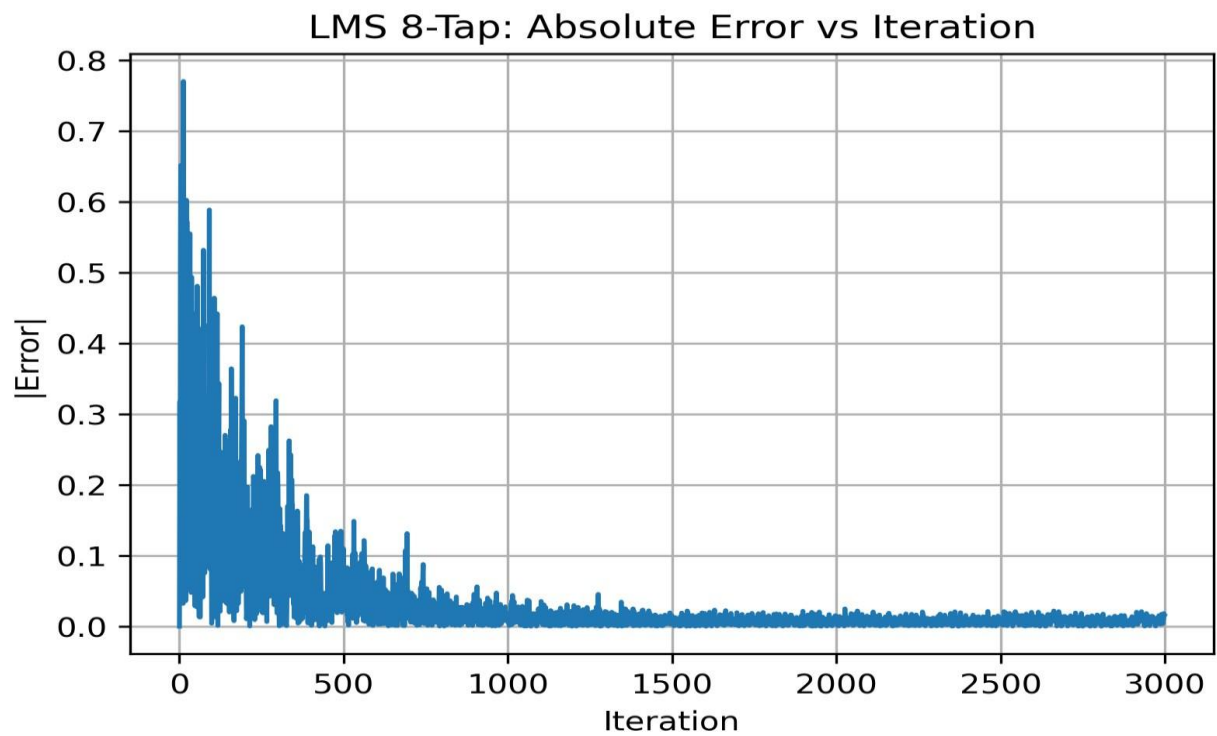


Fig 4. $|e(n)|$ vs Iteration for LMS

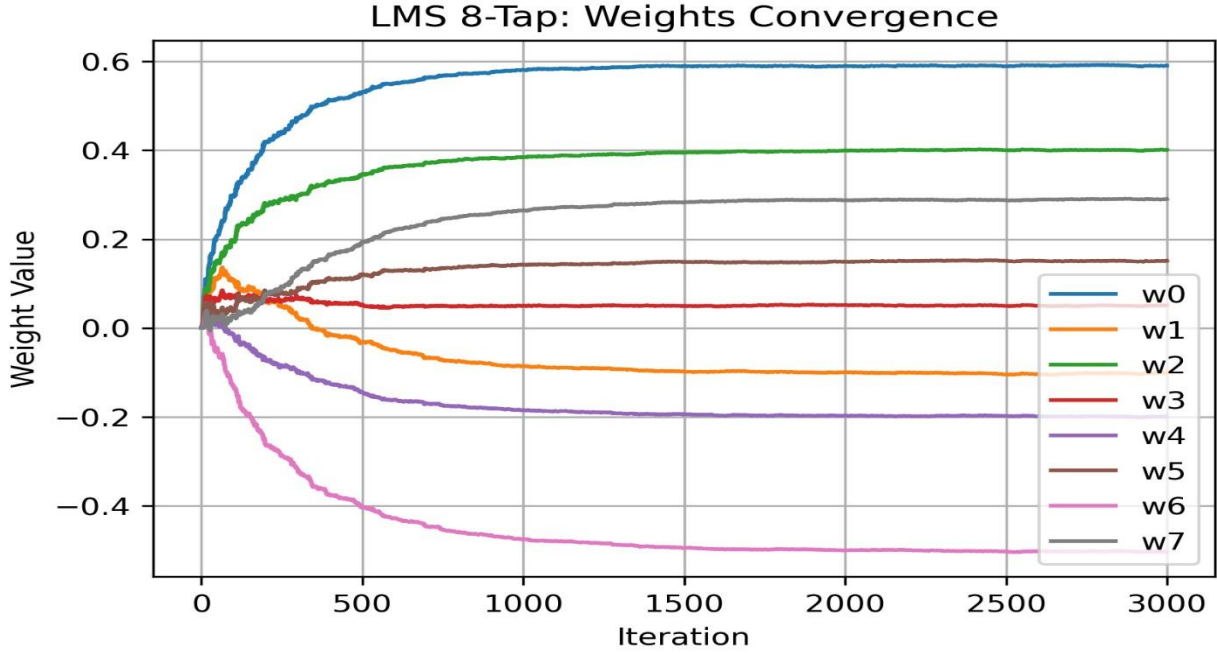


Fig 5. Convergence of weights for LMS

Synthesis Analysis: The design was synthesized in Vivado targeting a Xilinx FPGA. The synthesis report provides estimates of resource utilization and the **critical path delay**, which determines the maximum clock frequency (Fmax). The critical path in the LMS implementation traverses the FIR inner product accumulation, the error computation, and the gradient multiplication — confirming the feedback bottleneck identified in the architectural analysis.

Parameter	Value
Filter order	8
LUT count	144
Flip-Flop count	352
DSP blocks used	24
Critical path delay	21.8ns
Maximum clock frequency Fmax	45.87MHz

Table 2. Synthesis Results for LMS

The synthesis results establish a **performance baseline** against which the pipelined DLMS implementation will be compared in the following chapter. The critical path delay reported here directly motivates the need for pipelining: by inserting $L = 3$ pipeline registers along this path, DLMS breaks the combinational chain and achieves a significantly higher Fmax, at the cost of a controlled degradation in convergence speed analyzed in Chapter 4.

CHAPTER 4

Delayed LMS (DLMS) — Pipelining for Hardware Efficiency

4.1 Motivation for Pipelining in Adaptive Filters

In digital hardware design, **pipelining** is the most effective technique for increasing throughput. By inserting registers at intermediate points along a combinational datapath, a long computation is broken into shorter stages, each completing within a reduced clock period. The result is a higher achievable **Fmax** and, consequently, a higher sample processing rate without altering the final computed result.

For most digital signal processing operations, pipelining is straightforward. A **direct-form FIR filter**, for instance, can be freely pipelined by inserting registers between adder stages in the accumulation tree. Since the FIR computation has no feedback, the inserted registers simply introduce a fixed, predictable **output latency** without affecting correctness. The filter still produces the correct output it just does so a few clock cycles later.

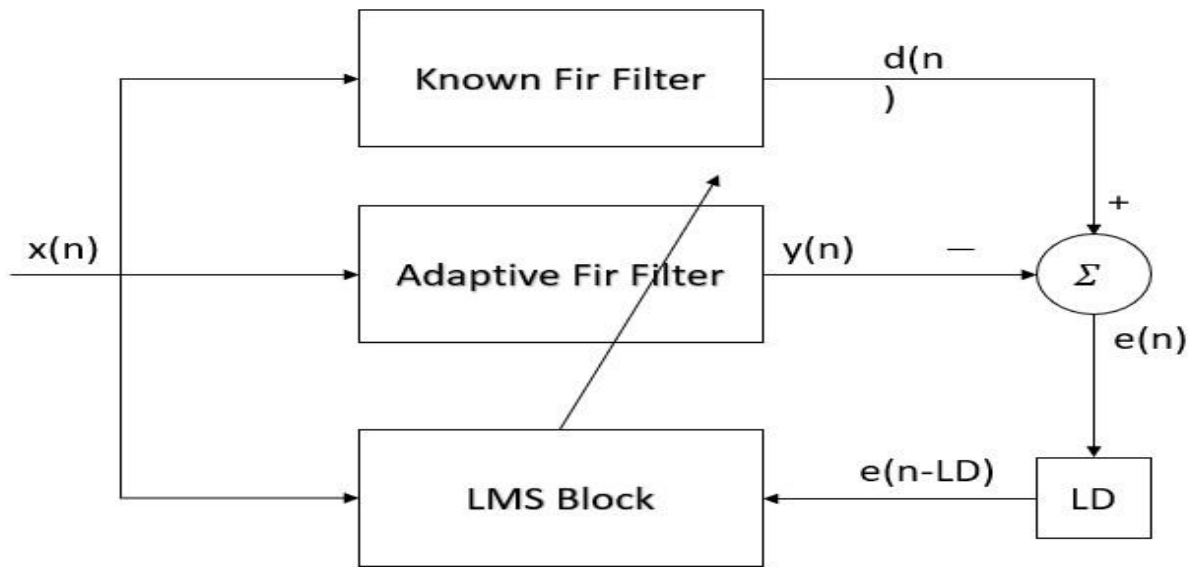


Fig 6. Block diagram for DLMS adaptive filter

The LMS adaptive filter, however, cannot be pipelined so freely. Its weight update equation creates a **loop-carried dependency**: the weight vector $w(n+1)$ is needed to compute $y(n+1)$, but $w(n+1)$ itself depends on $e(n)$, which depends on $y(n)$. This feedback loop means that every stage of the computation inner product, error, gradient, weight update must complete before the next

input sample can be correctly processed. The entire chain sits on the **critical path**, and any attempt to insert a pipeline register within this loop breaks the causal dependency between consecutive weight updates.

To quantify the problem: in the 8-tap Q4.12 LMS implementation of Chapter 3, the critical path traverses a 8-input multiply-accumulate chain, a subtraction for error computation, a gradient multiply, and a weight accumulation. The combined propagation delay of these operations limits the clock period, and therefore the Fmax, to a value well below what the FPGA fabric is capable of sustaining for simpler datapaths. This gap between achievable and theoretical Fmax represents wasted hardware potential.

The solution is not to simply remove the feedback the feedback is algorithmically necessary. Instead, the approach is to **restructure the algorithm itself** so that the feedback loop can tolerate inserted delays without losing correctness. This restructuring is formalized through **retiming theory** and leads directly to the **Delayed LMS (DLMS)** algorithm, which accepts an L-cycle delay in the weight update as a deliberate design choice in exchange for a fully pipelined, high-frequency hardware implementation.

4.2 DLMS Algorithm: Formulation and Retiming Theory

Retiming is a formal graph-theoretic transformation applicable to synchronous digital circuits. Given a circuit represented as a directed graph where nodes are computations and edges carry registers, retiming repositions registers along edges moving them forward or backward through nodes while preserving the circuit's input-output behavior at the boundaries. The key theorem underlying retiming is that register relocation does not alter the circuit's steady-state functional behavior, provided the total number of registers on every cycle in the graph is preserved.

Applied to the LMS feedback loop, retiming allows L pipeline registers to be inserted into the critical path. The consequence is that the weight used to compute $y(n)$ is not $w(n)$ but $w(n-L)$ the weight vector from L cycles earlier. If this delay is left unaccounted for, the algorithm diverges. The **Delayed LMS formulation** explicitly incorporates this delay into the update equation, making the algorithm aware of and consistent with the hardware's timing:

$$\begin{aligned} y(n) &= w^T(n) \cdot x(n) \\ e(n) &= d(n) - y(n) \end{aligned}$$

$$w(n+1) = w(n) + \mu \cdot e(n-L) \cdot x(n-L)$$

The weight update now uses the error and input from L cycles in the past, matching the delayed weight that was used to compute the output. This self-consistency is what makes DLMS algorithmically valid despite the introduced delay.

It has been shown that under stationary input conditions, DLMS converges to the same **WienerHopf optimal solution** w^* as standard LMS, provided the step size satisfies a modified stability bound:

$$0 < \mu < 2 / ((2L+1) \cdot \lambda_{\max})$$

The factor $(2L+1)$ in the denominator reflects the increased conservatism required as pipeline depth grows. For $L = 3$, this bound is approximately one-seventh of the standard LMS bound, meaning μ must be chosen smaller to maintain stability which in turn affects convergence speed.

4.2 Effect of Pipeline Depth L on Convergence

The pipeline depth L governs the fundamental trade-off in DLMS: **hardware speed versus algorithmic convergence**. As L increases, the critical path is subdivided into more stages, each with lower propagation delay, yielding a higher $F_{\max}$. However, the weight update becomes increasingly stale — it is correcting weights based on an error computed L cycles ago, during which the input has evolved and the optimal correction direction may have shifted.

The practical consequence is that DLMS requires more iterations to reach a given MSE level compared to standard LMS operating at the same step size. The **convergence rate** of DLMS, measured in iterations, degrades approximately as a function of the modified eigenvalue bound. For $L = 3$ — the value used in this work — the step size must be reduced relative to standard LMS to satisfy the tighter stability constraint, which is the primary driver of slower convergence rather than the delay itself.

Importantly, the **steady-state MSE** of DLMS is not inherently higher than that of LMS. If both algorithms are allowed to converge fully at their respective stable step sizes, the steady-state performance is determined by μ and input statistics, not by L . The degradation from DLMS is therefore primarily in **convergence speed** (more iterations needed) rather than in **final accuracy** (steady-state MSE), which is an acceptable trade-off in applications where throughput is the primary constraint.

For very large L ($L > 5$), the required step size becomes so small that convergence becomes impractically slow, negating the hardware benefit. The choice of $L = 8$ in this work represents a practical balance: it breaks the critical path into four stages, achieving a meaningful F_{max} improvement, while keeping the convergence degradation within an acceptable range.

4.4 Hardware Architecture and Verilog Implementation

The DLMS hardware architecture builds directly on the LMS design of Chapter 3, with two structural additions that implement the algorithmic delay: a **weight delay buffer** and an **errorinput delay buffer**.

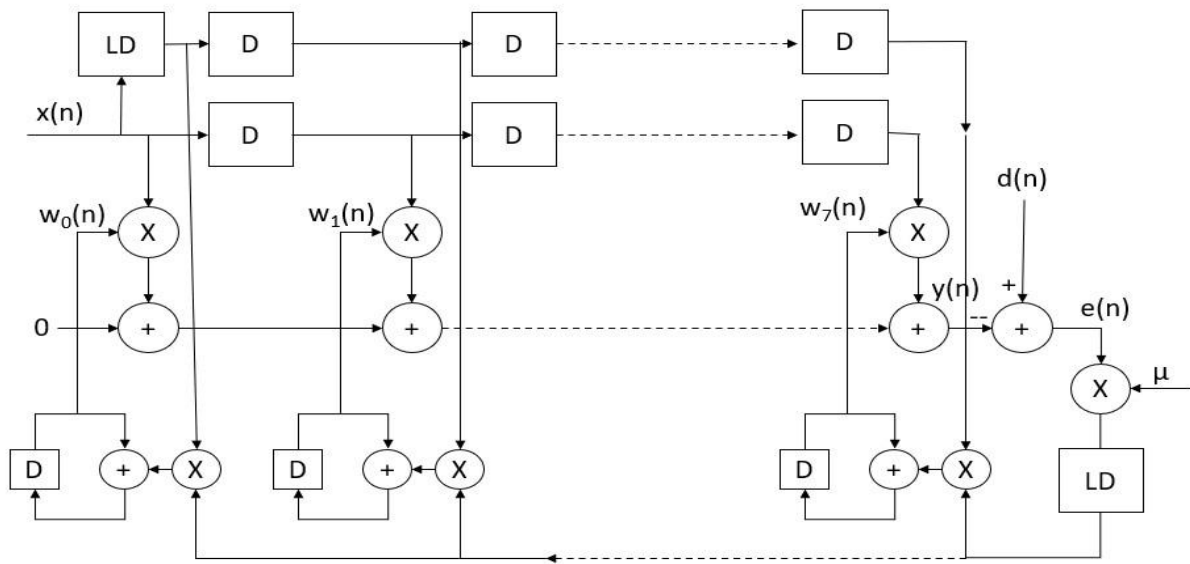


Fig 7. Architecture for DLMS adaptive filter

The **weight delay buffer** is implemented as an array of L register stages, each 16 bits wide and $N = 8$ elements deep, forming a shift register that holds the past L weight vectors. On each clock cycle, the weight vector propagates one stage through this buffer, so the FIR filter always computes its inner product using $w(n-L)$ — the weight vector from L cycles ago — rather than the most recently updated weights. This is the hardware realization of the delayed output equation $y(n) = w^T(n-L) \cdot x(n)$.

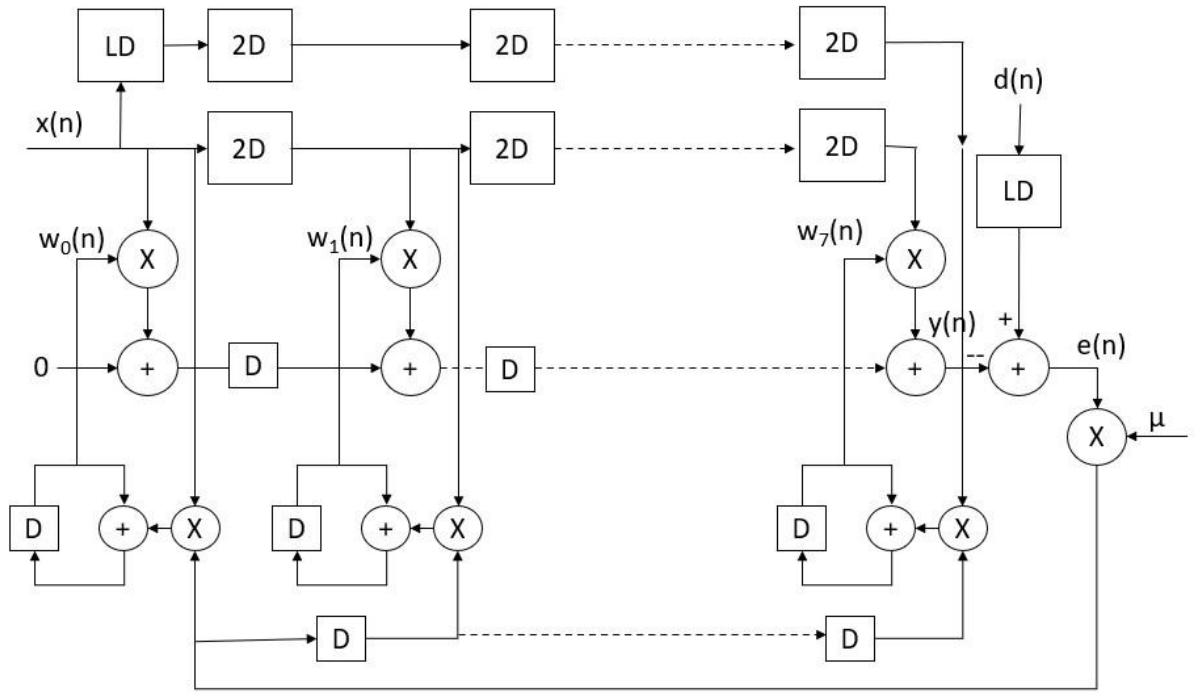


Fig 8. Retimed DLMS architecture

The **error-input delay buffer** holds the past L values of both $e(n)$ and $x(n)$, so the weight update receives $e(n-L)$ and $x(n-L)$. The update equation $w(n+1) = w(n) + \mu \cdot e(n-L) \cdot x(n-L)$ is then computed identically to the LMS update, with the only difference being that delayed signals feed into the gradient computation.

The critical path in DLMS no longer spans the full LMS feedback loop. With $L = 8$ pipeline registers inserted, the longest combinational path within a single clock cycle is reduced to approximately one-quarter of the original, directly enabling a proportionally higher clock frequency. The Verilog implementation uses **non-blocking assignments** throughout the registered pipeline stages and **generate loops** to instantiate the $N = 16$ parallel weight update units, each handling one tap independently.

4.5 Simulation Results and Synthesis Analysis

Simulation Setup: The DLMS filter was simulated under the same conditions as the LMS baseline identical sparse unknown system, identical input signal, identical testbench structure with the step size adjusted to satisfy the tighter DLMS stability bound for $L = 8$. This controlled setup ensures

that differences in the results are attributable solely to the effect of pipelining rather than to differing test conditions.

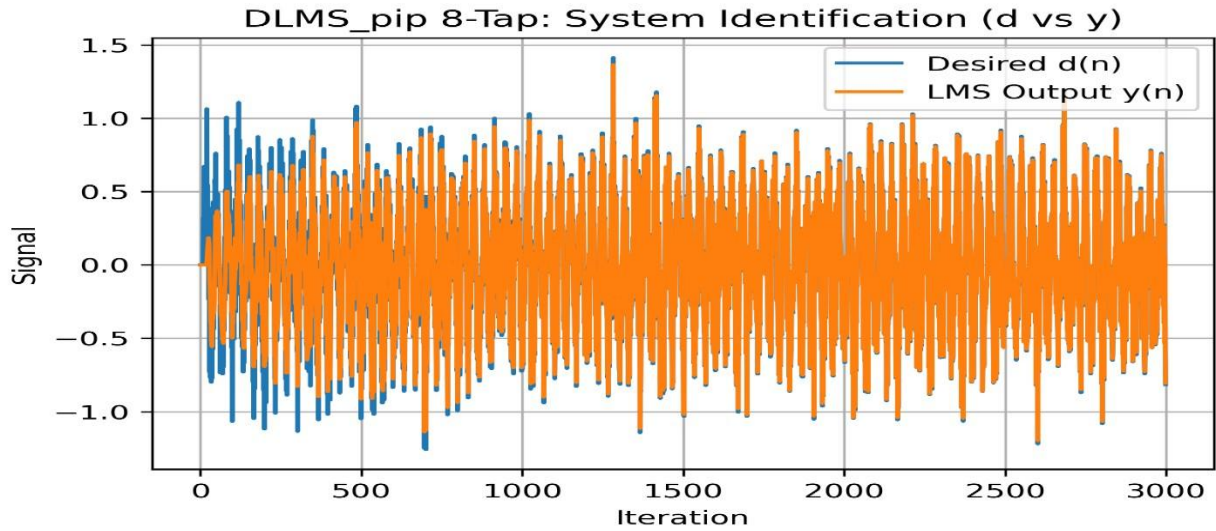


Fig 9. $y(n)$ vs $d(n)$ for retimed DLMS

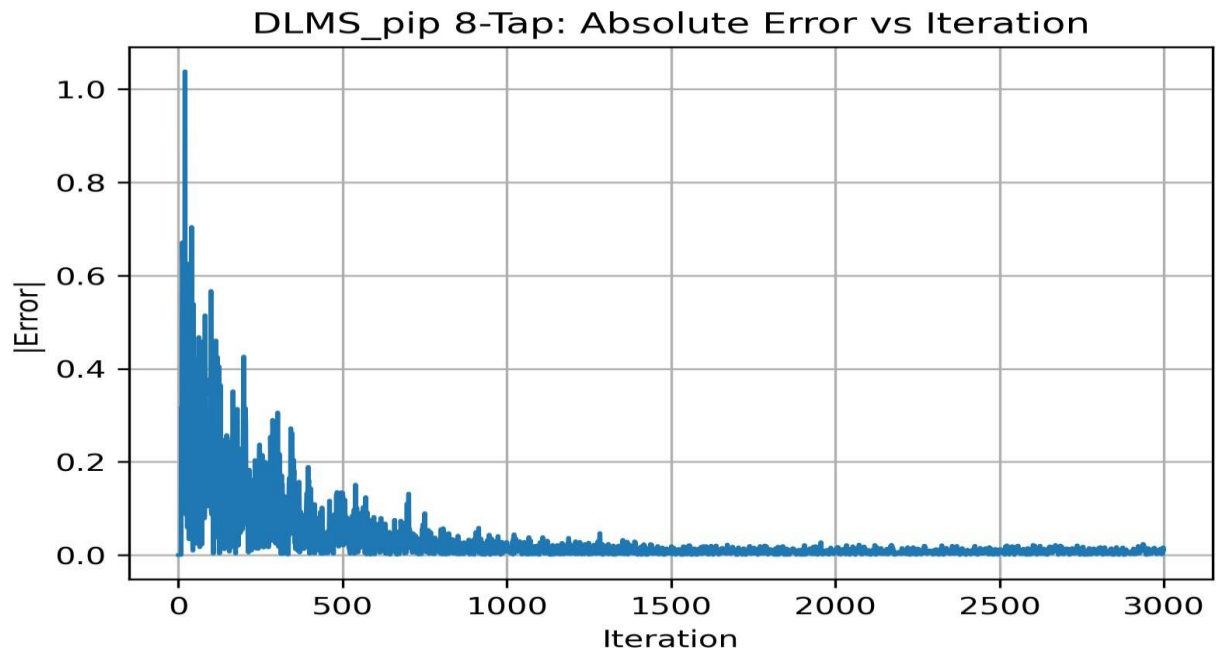


Fig 10. $|e(n)|$ vs Iteration for Retimed DLMS

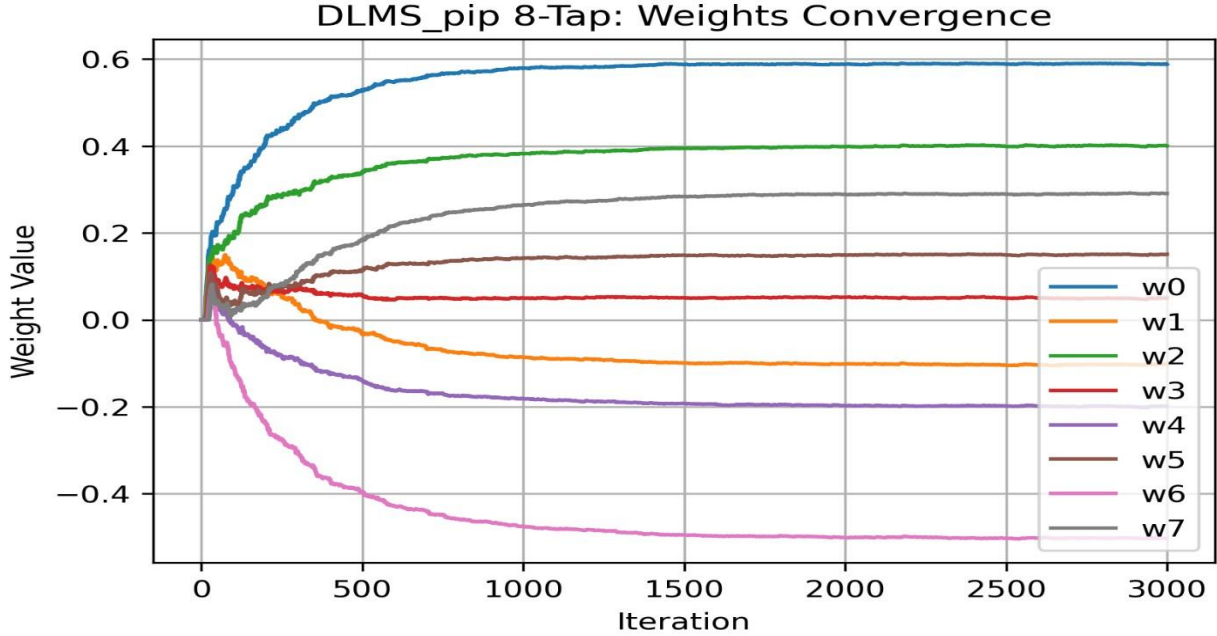


Fig 11. Convergence of weights for Retimed DLMS

Synthesis Analysis: Post-synthesis timing results confirm the expected improvement in Fmax relative to the baseline LMS implementation. The insertion of $L = 8$ pipeline stages breaks the critical combinational chain, reducing the worst-case path delay and allowing a higher clock frequency to be met.

Parameter	LMS	DLMS8_pip
Filter order	8	8
LUT count	144	184
Flip-Flop count	352	743
DSP blocks used	24	24
Critical path delay	21.8ns	13.6ns
Maximum clock frequency Fmax	45.87MHz	73.53MHz

Table 3. Synthesis results of Retimed DLMS and LMS

The increase in **flip-flop count** relative to LMS directly reflects the storage overhead of the weight delay buffer and error-input delay registers — an expected and acceptable cost. The **LUT and DSP counts** should remain comparable between LMS and DLMS, since the computational operations (multiply, accumulate, update) are identical; only the pipeline register structure differs. The Fmax improvement, expressed as a percentage gain over the LMS baseline, constitutes the primary hardware contribution of the DLMS implementation and quantifies the benefit of accepting the L-cycle convergence trade-off.

CHAPTER 5

ZA-LMS — Zero-Attracting LMS for Sparse Systems

5.1 Sparse System Identification and Motivation

In the **system identification** problem, the goal is to estimate the impulse response of an unknown system using only its input and output observations. Standard adaptive algorithms such as LMS treat this as a general estimation problem, making no assumptions about the structure of the unknown system. This generality is appropriate when the system's impulse response is **dense** that is, when most or all of its N coefficients carry significant energy.

However, many physical systems of practical interest are **sparse**: their impulse response contains only a small number of dominant non-zero coefficients, with the remainder being zero or negligibly small. Examples include **sparse multipath wireless channels**, where signal energy arrives at the receiver along only a few distinct propagation paths separated by long delays; **acoustic echo paths** in large rooms, where reflections are concentrated at specific time instants; and **network channel models** in digital subscriber lines, where the impulse response spans thousands of taps but only a handful are significant.

When standard LMS is applied to such a system, it expends equal adaptive effort on all N taps regardless of their true magnitude. The weight update drives every coefficient including the many that should remain at zero using the same gradient step. This results in **slow convergence** of the dominant taps and a **higher steady-state MSE** than is theoretically achievable, since the inactive taps continue to fluctuate around zero rather than being pinned there. For a 8-tap filter identifying a system where only 2 or 3 taps are non-zero, this inefficiency is significant.

The fundamental insight motivating sparsity-aware algorithms is that **prior knowledge of sparsity**, when incorporated into the update rule, can substantially improve convergence speed and steady-state accuracy without increasing computational complexity beyond $O(N)$.

5.2 ZA-LMS Algorithm: Formulation and Intuition

The **Zero-Attracting LMS (ZA-LMS)** algorithm incorporates sparsity into the LMS framework by augmenting the standard weight update with a **regularization term** derived from the **L1 norm** of the weight vector. The L1 norm, defined as the sum of absolute values of all weights, serves as

a **convex sparsity-promoting penalty** — minimizing it encourages as many weights as possible to be exactly zero, which aligns with the physical structure of the sparse unknown system.

The ZA-LMS update equation is:

$$w(n+1) = w(n) + \mu \cdot e(n) \cdot x(n) - \rho \cdot \text{sgn}(w(n))$$

where ρ is the **zero-attraction coefficient**, a small positive scalar that controls the strength of the sparsity-promoting term, and $\text{sgn}(\cdot)$ is the element-wise sign function applied to the weight vector:

$$\text{sgn}(w_k) = +1 \text{ if } w_k > 0, -1 \text{ if } w_k < 0, 0 \text{ if } w_k = 0$$

The term $-\rho \cdot \text{sgn}(w(n))$ acts as a **soft-thresholding correction**: it subtracts a fixed quantity ρ from each positive weight and adds ρ to each negative weight, pulling every coefficient toward zero by a constant amount at every iteration. For weights that are already near zero corresponding to the inactive taps of the sparse system this attraction is sufficient to drive them to zero and keep them there. For weights with large magnitude corresponding to the dominant taps the attraction term ρ is small relative to the gradient correction $\mu \cdot e(n) \cdot x(n-k)$, so it causes only a minor perturbation to their convergence trajectory.

This formulation makes ZA-LMS algorithmically almost identical to LMS in structure the only addition is the sign computation and the scalar subtraction per tap making it particularly attractive from a hardware implementation standpoint.

5.3 Effect of the Zero-Attraction Parameter ρ

The zero-attraction coefficient ρ is the key tuning parameter of ZA-LMS, and its selection involves a trade-off analogous to the step-size selection in standard LMS. Specifically, ρ controls the balance between **sparsity exploitation** and **convergence accuracy** for the dominant coefficients.

When ρ is too small, the zero-attraction term has negligible effect on the inactive taps. The algorithm behaves essentially identically to standard LMS, offering no practical improvement on sparse systems. The inactive weights continue to fluctuate around zero under the influence of the gradient noise, without being pulled down decisively.

When ρ is too large, the zero-attraction term dominates the update for all coefficients, including the dominant ones. Large weights are pulled toward zero too aggressively, interfering with the gradient-driven convergence of the significant taps. This manifests as a **bias in the converged**

weight estimates the dominant coefficients settle at values lower than their true magnitudes, increasing the steady-state MSE. In extreme cases, a very large ρ can cause all weights to collapse to zero, leading to complete failure of adaptation.

The optimal ρ lies between these extremes, chosen such that the attraction term is strong enough to decisively zero out the inactive taps within a reasonable number of iterations, while remaining small enough to cause only negligible perturbation to the dominant tap trajectories. In practice, ρ is selected empirically by observing the convergence curve on a representative sparse channel. A useful heuristic is that ρ should be at least an order of magnitude smaller than $\mu \cdot \sigma_e \cdot \sigma_x$, so that the sparsity correction does not overwhelm the gradient step.

In the hardware implementation, ρ is stored as a fixed-point integer in Q4.12 format, analogous to μ . The sign computation is performed directly on the two's complement representation of each weight extracting the MSB gives the sign bit making the zero attractor unit computationally trivial to implement in Verilog.

5.4 Hardware Architecture and Verilog Implementation

The hardware architecture of ZA-LMS is an extension of the LMS architecture described in Chapter 3. The FIR filter core and the error computation block remain structurally identical. The modification is confined entirely to the **weight update core**, where an additional **Zero Attractor Unit (ZAU)** is introduced alongside the standard gradient computation.

The **Zero Attractor Unit** operates as follows for each tap k . Given the current weight $w_k(n)$ stored as a signed 16-bit Q4.12 value, the sign is extracted directly from the **most significant bit (MSB)** of the two's complement representation. If the MSB is 1, the weight is negative and the attraction term is $+\rho$; if the MSB is 0 and the weight is non-zero, the attraction term is $-\rho$; if the weight is exactly zero, the attraction is zero. This three-way conditional maps cleanly onto a small combinational logic block requiring no multipliers only a comparison and a multiplexer.

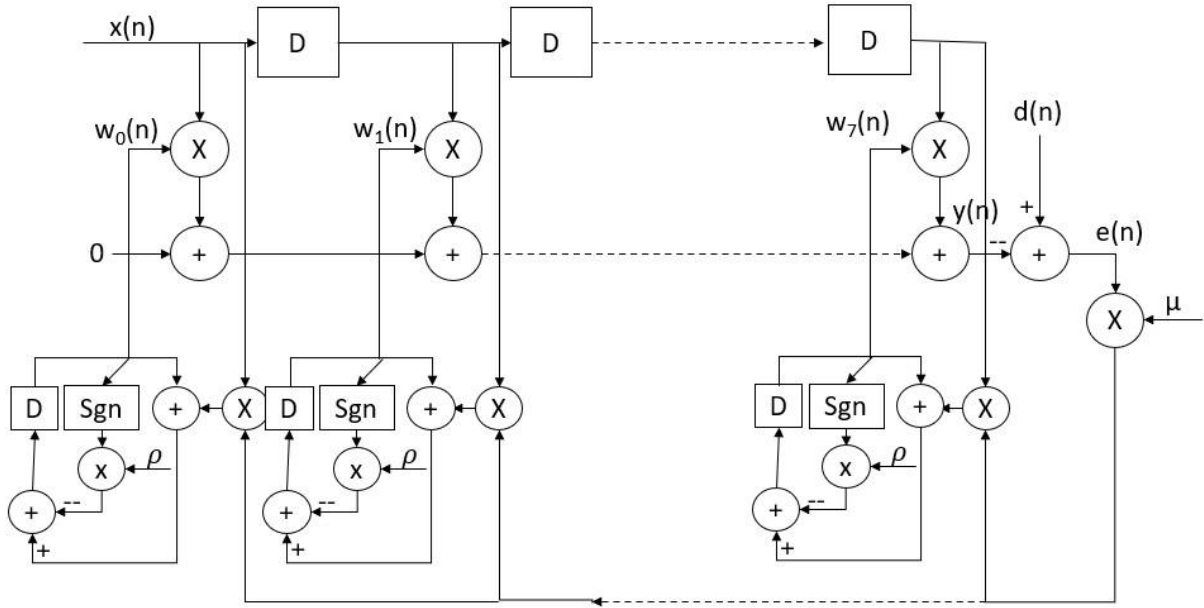


Fig 12. Architecture of ZA-LMS adaptive filter The complete weight update for each tap in Verilog is:

$$w_k(n+1) = w_k(n) + \mu \cdot e(n) \cdot x(n-k) - \rho \cdot \text{sgn}(w_k(n))$$

where all three terms are computed in parallel for all 8 taps simultaneously. The gradient term $\mu \cdot e(n) \cdot x(n-k)$ is computed identically to the LMS implementation, and the zero-attraction term $\rho \cdot \text{sgn}(w_k(n))$ is added as a separate correction path. Both corrections are accumulated into the weight register using a single adder, keeping the update atomic within one clock cycle.

The **critical path** in ZA-LMS is marginally longer than in LMS due to the additional adder for combining the two correction terms. However, since the ZAU itself is purely combinational logic with no multipliers, this addition is minimal and does not significantly degrade the achievable $F_{\max}$ relative to standard LMS.

5.5 Simulation Results and Synthesis Analysis

Simulation Setup: The ZA-LMS filter was simulated on the same sparse system identification scenario used for LMS and DLMS a 16-tap unknown system with a small number of dominant non-zero coefficients and the remainder at zero. Identical input sequences and desired signals were used to ensure comparability. The step size μ was set to the same value as in the LMS simulation,

and the zero-attraction coefficient ρ was tuned to an appropriate value within the stable operating region identified in Section 5.3.

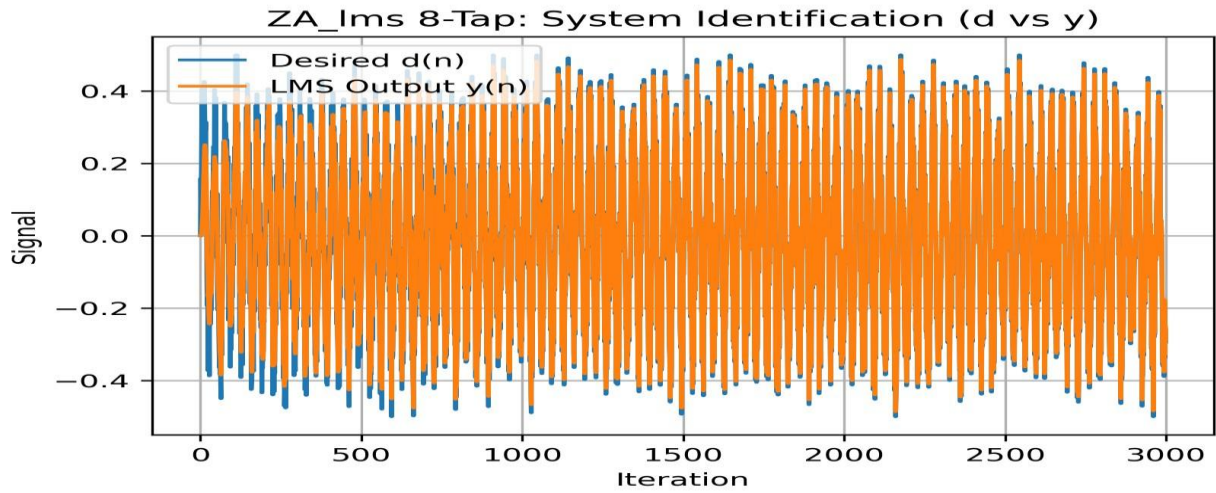


Fig 13. $y(n)$ vs $d(n)$ for ZA-LMS

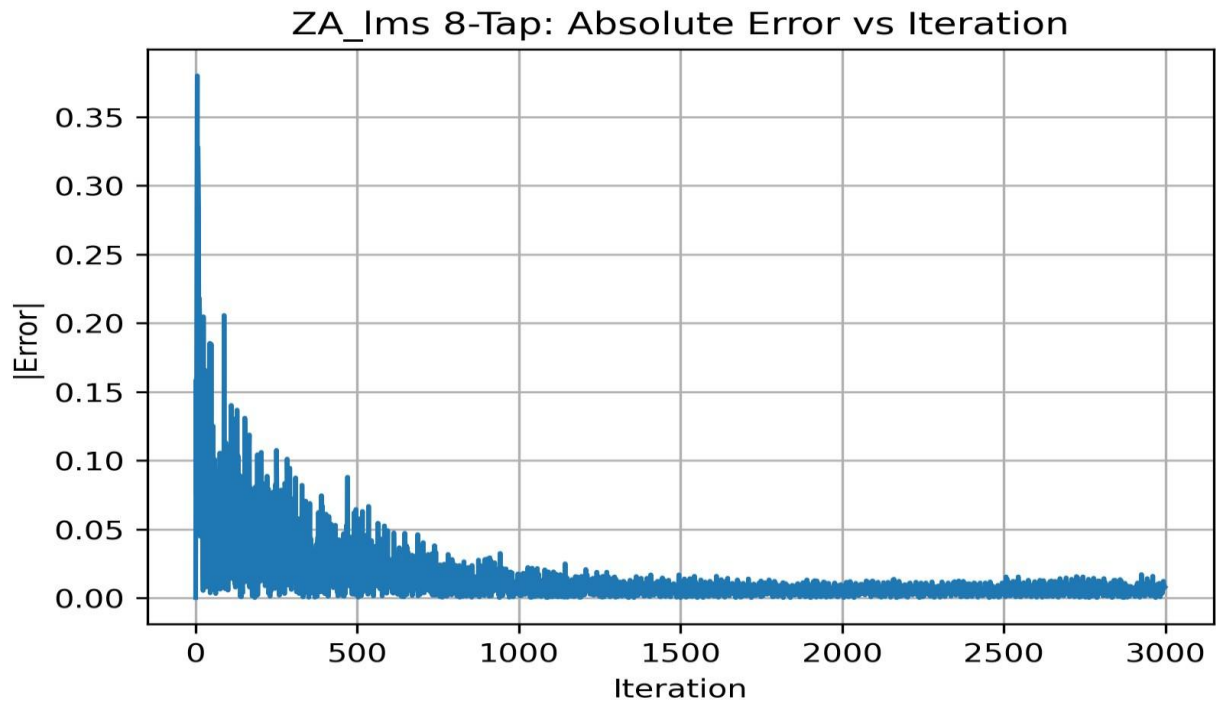


Fig 14. $|e(n)|$ vs iteration for ZA-LMS

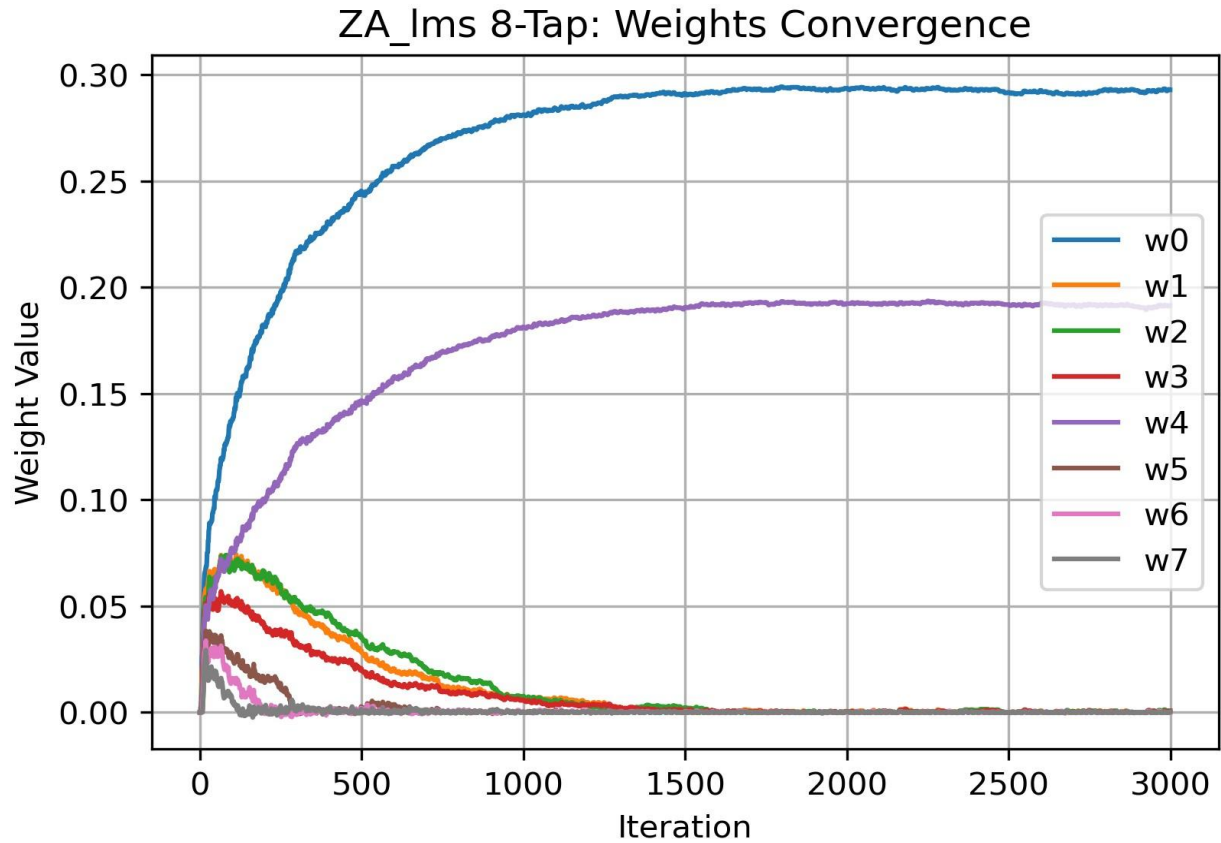


Fig 15. Convergence of weights for ZA-LMS

Synthesis Analysis: The ZA-LMS design was synthesized in Vivado under the same target constraints as the LMS implementation. The resource overhead of the Zero Attractor Unit is expected to be modest, since the sign extraction and conditional assignment require only combinational logic with no DSP block usage.

Parameter	Value
Filter order	8
LUT count	208
Flip-Flop count	304
DSP blocks used	18
Critical path delay	21.8ns
Maximum clock frequency Fmax	45.87MHz

Table 4. Synthesis results for ZA-LMS

Parameter	Value
Filter order	8

LUT count	247
Flip-Flop count	693
DSP blocks used	18
Critical path delay	7.1ns
Maximum clock frequency Fmax	140.84MHz

Table 5. Synthesis results for ZA-LMSpip

The synthesis results are interpreted in the context of the LMS baseline. Any increase in LUT count directly reflects the combinational overhead of the ZAU. The critical path delay is expected to be comparable to LMS, since the ZAU does not introduce any multiplications into the update path. A complete side-by-side comparison of ZA-LMS against LMS and DLMS including convergence speed, steady-state MSE, Fmax, and resource cost is deferred to Chapter 7, where all four implementations are evaluated under unified experimental conditions.

CHAPTER 6

RZA-LMS — Reweighted Zero-Attracting LMS

6.1 Sparse Adaptive Filtering and Limitations of ZA-LMS

The **Zero-Attracting LMS** algorithm, developed in Chapter 5, demonstrated a meaningful improvement over standard LMS for sparse system identification by incorporating an L1 norm penalty into the weight update. However, the mechanism by which ZA-LMS promotes sparsity subtracting a **constant** quantity $\rho \cdot \text{sgn}(w_k)$ from every weight at every iteration contains a fundamental limitation that becomes apparent under closer examination.

The zero-attraction term in ZA-LMS is **non-selective**: it applies the same magnitude of attraction ρ to every coefficient, regardless of whether that coefficient is large, small, or already at zero. For the inactive taps of a sparse system, this constant pull is exactly what is needed it drives small weights decisively to zero. However, for the **dominant taps** carrying the true system energy, the same attraction of magnitude ρ acts as a persistent bias, pulling the weight away from its true optimal value at every iteration. The converged weight estimate is therefore not the true Wiener solution but a **biased approximation**, with each dominant coefficient underestimated by an amount proportional to ρ .

This bias is the central weakness of ZA-LMS. In highly sparse systems where ρ must be set large enough to rapidly zero out many inactive taps, the resulting bias on the few dominant taps is nonnegligible and directly raises the steady-state **MSE floor** above what is theoretically achievable. The algorithm effectively over-penalizes the significant coefficients in its effort to suppress the insignificant ones.

The **Reweighted Zero-Attracting LMS (RZA-LMS)** algorithm was developed to resolve this contradiction. By making the attraction strength **inversely proportional to each weight's magnitude**, RZA-LMS applies strong attraction only to near-zero coefficients while leaving dominant coefficients essentially undisturbed achieving selective sparsity exploitation without the bias penalty.

6.2 RZA-LMS Algorithm: Formulation and Intuition

RZA-LMS replaces the uniform sign-based attraction of ZA-LMS with a **reweighted** attraction term derived from a **reweighted L1 norm** penalty. The update equation is:

$$w(n+1) = w(n) + \mu \cdot e(n) \cdot x(n) - \rho \cdot \text{sgn}(w(n)) / (1 + \varepsilon \cdot |w(n)|)$$

where the element-wise operation for tap k is:

$$w_k(n+1) = w_k(n) + \mu \cdot e(n) \cdot x(n-k) - \rho \cdot \text{sgn}(w_k(n)) / (1 + \varepsilon \cdot |w_k(n)|)$$

Here, ρ retains its role as the zero-attraction coefficient, and ε is a small positive **regularization constant** introduced to prevent division by zero when a weight is exactly zero and to control the transition between strong and weak attraction regions.

The key term is the denominator $(1 + \varepsilon \cdot |w_k(n)|)$. When $|w_k|$ is small — as is the case for inactive taps near zero — the denominator approaches 1, and the attraction reduces to $\rho \cdot \text{sgn}(w_k)$, identical to ZA-LMS. When $|w_k|$ is large — as is the case for dominant taps — the denominator becomes large, and the attraction term $\rho / (1 + \varepsilon |w_k|)$ approaches zero. The dominant coefficients therefore experience **negligible zero-attraction**, converging to their true Wiener values without the bias that afflicts ZA-LMS.

Intuitively, RZA-LMS behaves like a **spatially adaptive** version of ZA-LMS: each tap receives a customized level of zero-attraction based on its current magnitude, with the attraction automatically concentrating on the taps that are most likely inactive. This selectivity is what drives the convergence advantage of RZA-LMS over ZA-LMS on sparse systems.

6.3 Effect of Parameters ρ and ε on Convergence

RZA-LMS introduces two sparsity-related parameters — ρ and ε — each affecting convergence in distinct ways, and their joint selection requires careful consideration.

The role of ρ in RZA-LMS is broadly analogous to its role in ZA-LMS: it sets the overall magnitude of the zero-attraction force. A larger ρ drives inactive taps to zero more aggressively, improving convergence speed on sparse systems but potentially introducing residual bias if set too high. However, because the denominator $(1 + \varepsilon |w_k|)$ scales down the attraction for large weights, the bias on dominant taps in RZA-LMS is significantly lower than in ZA-LMS for the same ρ .

This means RZA-LMS can tolerate a larger ρ — and therefore achieve faster zeroing of inactive taps — without incurring the same penalty on dominant coefficients.

The **regularization constant** ε controls the sharpness of the transition between the strong attraction region (small weights) and the weak-attraction region (large weights). A large ε makes the denominator grow quickly with weight magnitude, concentrating the attraction very tightly around zero — ideal for highly sparse systems where the dominant coefficients are well-separated in magnitude from the inactive ones. A small ε broadens the transition region, making the algorithm behave more like ZA-LMS in systems with moderate sparsity. Setting $\varepsilon = 0$ recovers ZA-LMS exactly, confirming that ZA-LMS is a special case of the RZA-LMS framework.

In the fixed-point hardware implementation, both ρ and ε are stored as Q4.12 integers. The product $\varepsilon \cdot |w_k(n)|$ must be computed and added to the constant 1, which is represented as 4096 in Q4.12. The selection of ε must also account for fixed-point precision — a value of ε too small to be represented accurately in Q4.12 will effectively behave as zero, collapsing RZA-LMS to ZA-LMS in hardware even if the intended design is different.

6.4 Hardware Architecture and Verilog Implementation

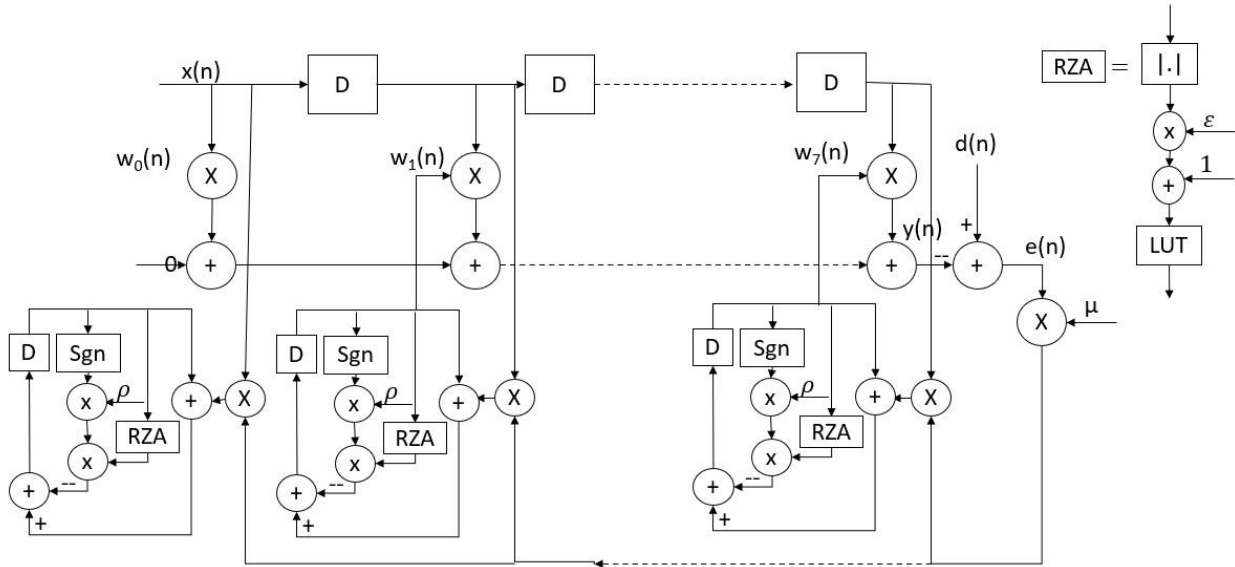


Fig 16. Architecture of RZA-LMS

The hardware architecture of RZA-LMS follows the same overall organization as ZA-LMS, with the FIR filter core and error computation block unchanged. The modification relative to ZA-

LMS is entirely within the weight update core, where the **Zero Attractor Unit (ZAU)** is replaced by the more complex **Reweighted Zero Attractor Unit (RZAU)**.

The computationally demanding element of the RZAU is the **reciprocal computation** required to evaluate $1 / (1 + \varepsilon \cdot |w_k|)$. Unlike the sign extraction in ZA-LMS, which is a trivial one-bit operation, the reciprocal is a non-trivial arithmetic function with no direct equivalent in standard combinational logic. Several approaches are available for fixed-point reciprocal approximation in hardware.

The most common approach for FPGA implementation is a **lookup table (LUT)-based approximation**, where the denominator $(1 + \varepsilon|w_k|)$ is used as an address into a pre-computed reciprocal table stored in block RAM or distributed LUTs. This requires quantizing the denominator to a manageable number of address bits — typically 8 to 10 — and accepting the approximation error introduced by this quantization. An alternative is **iterative NewtonRaphson refinement**, which starts from an initial approximation and converges to the true reciprocal in two or three iterations, but this introduces latency that must be pipelined carefully.

In this implementation, a **first-order LUT approximation** was adopted. The denominator is truncated to its upper 8 bits after computing $(1 + \varepsilon|w_k|)$ in Q4.12, and the corresponding reciprocal is retrieved from a 256-entry pre-computed table. The approximation error introduced by this truncation is analyzed in Section 6.5. All 16 taps are processed in parallel, with one RZAU instantiated per tap using a Verilog **generate loop**.

6.5 Simulation Results and Synthesis Analysis

Simulation Setup: The RZA-LMS filter was simulated on the same sparse system identification scenario used throughout Chapters 3 through 5, ensuring direct comparability. The step size μ was held at the same value used for ZA-LMS, and the parameters ρ and ε were tuned within their respective stable regions. Given the added complexity of the RZAU — particularly the LUT-based reciprocal approximation — the simulation also serves to verify that the fixed-point approximation does not introduce instability or systematic bias into the weight update.

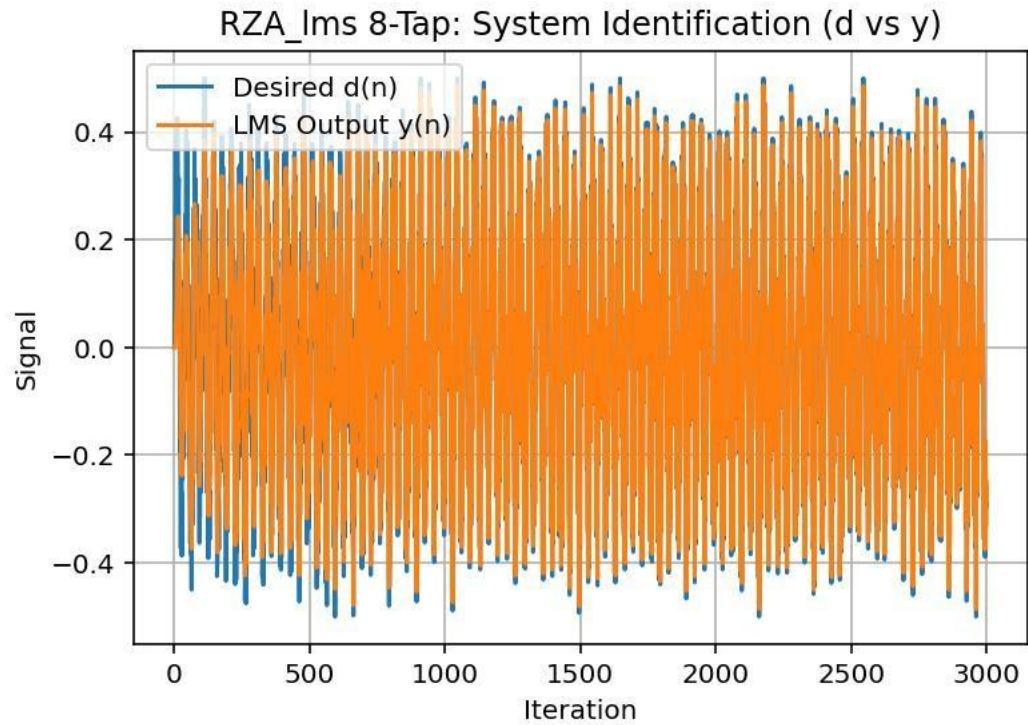


Fig 17. $y(n)$ vs $d(n)$ for RZA-LMS

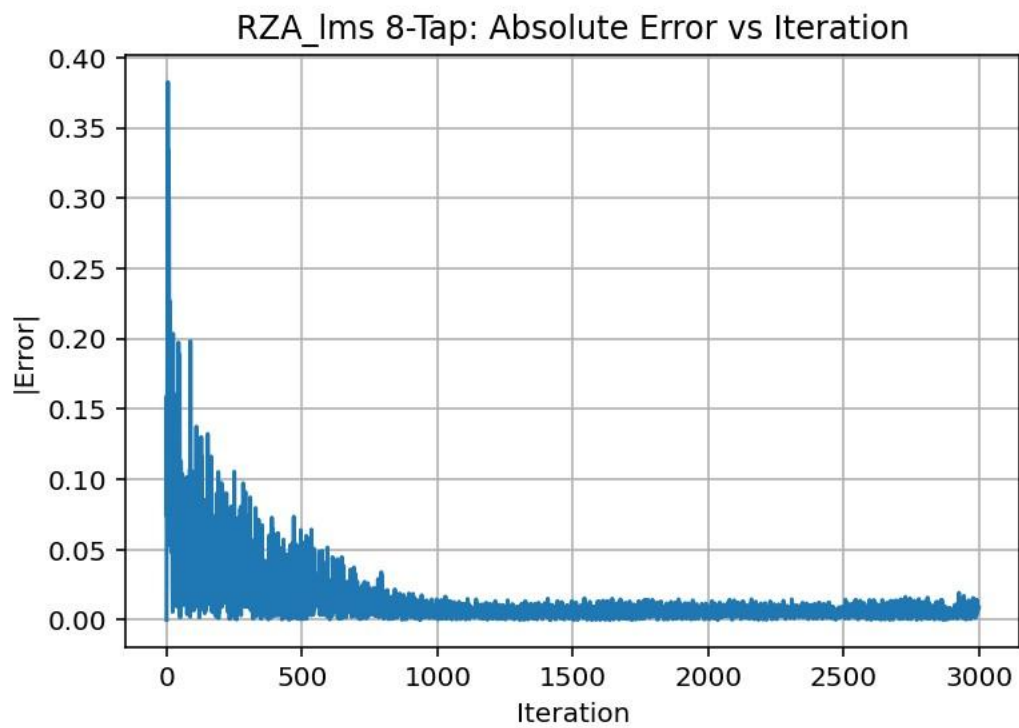


Fig 18. $|e(n)|$ vs iteration for RZA-LMS

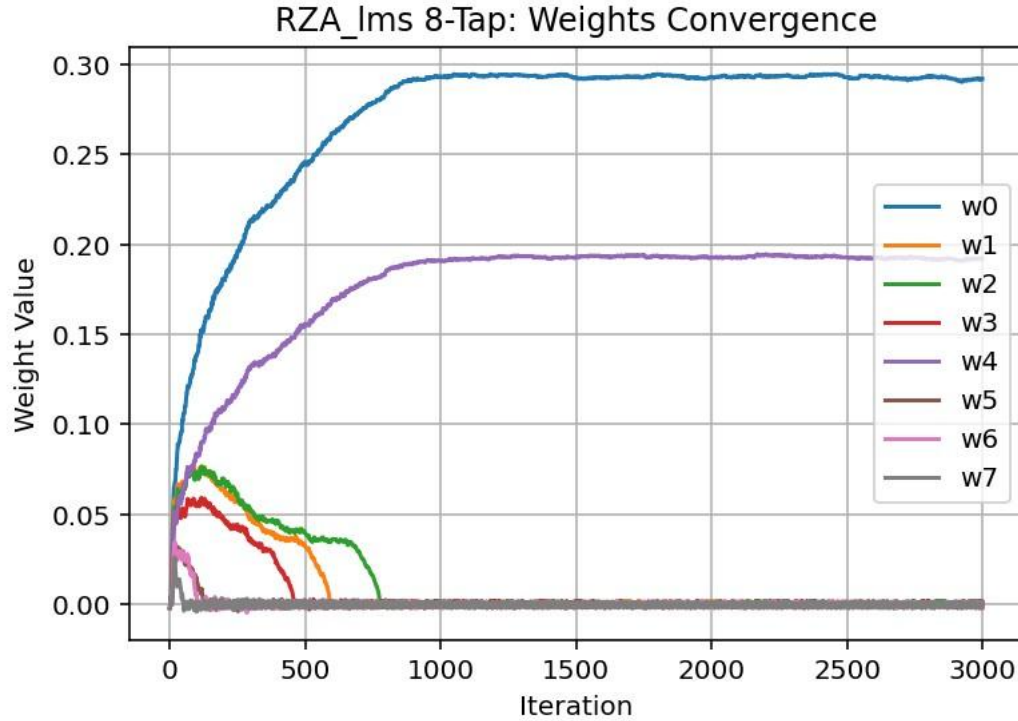


Fig 19. Convergence of weights for RZA-LMS

Synthesis Analysis: RZA-LMS carries a higher hardware cost than ZA-LMS due to the RZAU's absolute value computation, multiplication by ϵ , denominator accumulation, and LUT-based reciprocal lookup. These operations consume additional LUTs and may introduce a longer critical path than either LMS or ZA-LMS.

Parameter	Value
Filter order	8
LUT count	152
Flip-Flop count	304
DSP blocks used	18
Critical path delay	21.9ns
Maximum clock frequency Fmax	45.66MHz

Table 6. Synthesis results for RZA-LMS

Parameter	Value
-----------	-------

Filter order	8
LUT count	191
Flip-Flop count	693
DSP blocks used	18
Critical path delay	7.4ns
Maximum clock frequency Fmax	135.13MHz

Table 7. Synthesis results for RZA-LMSpip

The synthesis results are discussed in the context of the added architectural complexity. Any degradation in Fmax relative to ZA-LMS is attributed to the RZAU critical path, and any increase in LUT usage reflects the reciprocal approximation table and the additional arithmetic logic. A full algorithmic and hardware comparison of RZA-LMS against all three prior implementations is presented in Chapter 7.

CHAPTER 7

Comparative Results and Analysis

7.1 Experimental Setup and Parameters

To ensure that the comparative analysis is meaningful and free from confounding variables, all four adaptive filter implementations — **LMS**, **DLMS**, **ZA-LMS**, and **RZA-LMS** — were evaluated under a common, controlled experimental framework. The simulation environment, input signal characteristics, unknown system model, and hardware synthesis constraints were held identical across all implementations, so that observed differences in performance are attributable solely to algorithmic and architectural distinctions.

Algorithm	Step size μ	Sparsity parameters	Conv.iteration	SteadyStateMSE(db)
LMS	1/32		1072	-40.43
DLMSp	1/32		1069	-40.45
ZA-LMS	1/32	$\rho = 1/4096$,	991	-42.99
RZA-LMS	/32	$\rho = 1/4096$, $\varepsilon \approx 0.0625$	882	-42.92

Table 8. Parameter's used

The unknown system used throughout all experiments is a **8-tap sparse FIR system** with a small number of non-zero dominant coefficients and the remainder at zero. The input signal $x(n)$ is a **zero-mean white Gaussian noise** sequence generated in Python and applied identically to all four testbenches. The desired signal $d(n)$ is the convolution of $x(n)$ with the true sparse system impulse response, with a small amount of additive observation noise included to simulate a realistic identification scenario. All signals are quantized to **Q4.12 fixed-point format** before being applied to the Verilog testbenches.

The step size μ for LMS and ZA-LMS was set to the same value, within the standard LMS stability bound. For DLMS with $L = 8$, μ was reduced to satisfy the tighter modified stability bound. For RZA-LMS, μ was matched to ZA-LMS. The sparsity parameters ρ and ε for ZA-LMS and RZALMS were individually tuned to their operating points identified in Chapters 5 and 6 respectively.

All synthesis runs used identical clock constraint and target device settings in Vivado.

7.2 Convergence Comparison: LMS, DLMS, ZA-LMS, RZA-LMS

The convergence performance of all four algorithms is compared along two primary dimensions: **convergence speed** (rate of MSE reduction during the transient phase) and **steady-state MSE** (the noise floor reached after convergence).

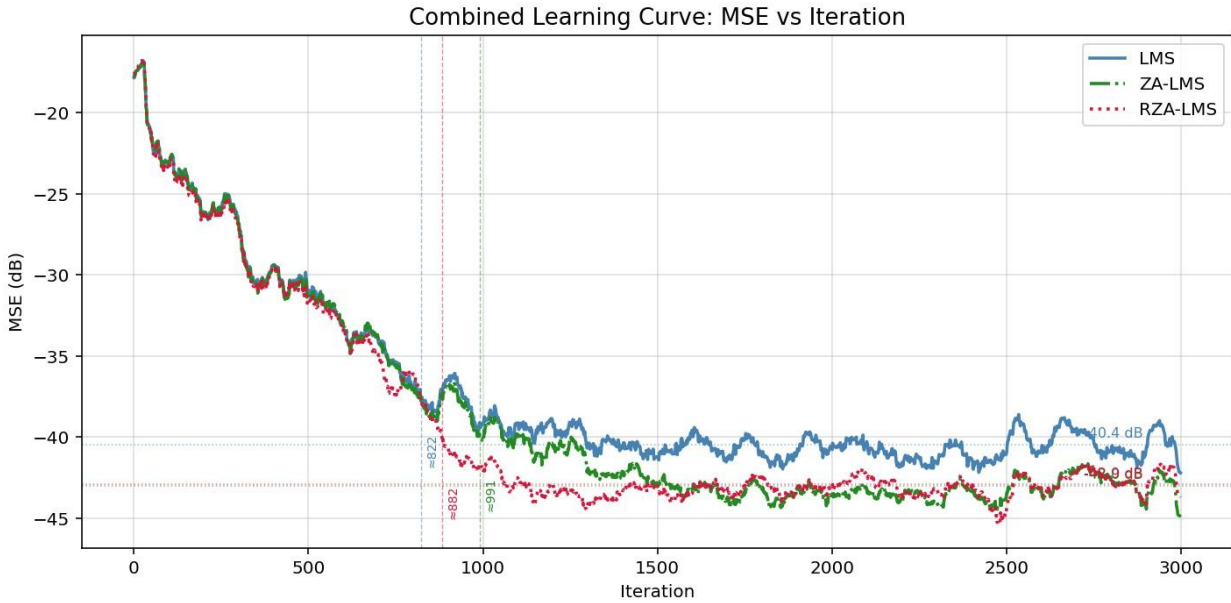


Fig 20. MSE vs Iteration comparison

LMS serves as the baseline. It converges steadily under the standard gradient descent mechanism but makes no use of the system's sparsity, resulting in the slowest convergence among the sparsityaware algorithms and the highest steady-state MSE on the sparse channel.

DLMS exhibits slower convergence than LMS in terms of iteration count, a direct consequence of the reduced step size required by the $L = 8$ stability constraint. Its steady-state MSE is comparable to LMS since both algorithms converge to the same Wiener solution. The trade-off accepted by DLMS is therefore purely in the convergence speed dimension, in exchange for the hardware timing benefit demonstrated in Section 7.3.

ZA-LMS shows improved convergence speed over LMS on the sparse channel, as the zeroattraction term accelerates the zeroing of inactive taps and focuses the gradient energy on the dominant coefficients. The steady-state MSE is lower than LMS, confirming that sparsity exploitation yields a meaningful accuracy improvement. However, the residual bias on dominant

coefficients — discussed in Chapter 6 — is visible as a slightly elevated steady-state floor relative to RZA-LMS.

RZA-LMS achieves the best convergence performance among all four algorithms on the sparse channel, combining fast transient convergence with the lowest steady-state MSE. The reweighted attraction mechanism eliminates the dominant-tap bias of ZA-LMS while maintaining aggressive zeroing of inactive taps. The improvement over ZA-LMS is most pronounced at steady state, where the bias reduction directly translates to a lower MSE floor.

7.3 Synthesis Comparison: Timing and Resource Utilization

The hardware performance of all four implementations is compared using **post-synthesis reports** generated by Vivado under identical target device and clock constraint settings. The metrics of interest are: **critical path delay** (which determines the maximum achievable clock frequency F_{max}), **LUT utilization** (combinational logic cost), **flip-flop count** (register storage cost), and **DSP block usage** (dedicated multiplier resources).

Parameter	LMS	DLMS8_pip	ZA_LMS/pip	RZA_LMS/pip
Filter order	8	8	8	8
LUT count	144	184	208/247	152/191
Flip-Flop count	352	743	304/693	304/693
DSP blocks used	24	24	18	18
Critical path delay	21.8ns	13.6ns	21.8/7.1ns	21.9/7.4ns
Maximum frequency F_{max}	clock 45.87MHz	73.53MHz	45.87/140.84MHz	45.66/135.13MHz

Table 9. Synthesis results comparison

Several observations follow directly from the synthesis results. The **LMS** implementation establishes the resource and timing baseline. Its critical path spans the full multiply-accumulate-error-gradient-update chain, giving the longest combinational delay and therefore the lowest F_{max} among all implementations.

DLMS achieves the highest F_{max} among the four designs. By inserting $L = 3$ pipeline registers along the critical feedback path, the worst-case combinational delay per clock cycle is reduced to approximately one-quarter of the LMS critical path. The cost of this improvement is visible in the flip-flop count, which increases due to the weight delay buffer and error-input delay registers. LUT and DSP usage remain comparable to LMS since the underlying arithmetic operations are identical.

ZA-LMS introduces a modest LUT overhead relative to LMS, attributable to the combinational logic of the Zero Attractor Unit — sign extraction, conditional assignment, and the additional adder in the weight update path. The critical path is marginally longer than LMS but does not significantly degrade Fmax, confirming that the ZAU is not the timing bottleneck. DSP usage is unchanged since no additional multiplications are introduced.

RZA-LMS carries the highest resource cost among all four implementations, as expected from the complexity of the Reweighted Zero Attractor Unit. The LUT-based reciprocal approximation, absolute value computation, and additional multiplications consume measurably more LUTs and potentially additional DSP blocks. The critical path through the RZAU may also be longer than in ZA-LMS, yielding a lower Fmax. This resource-performance trade-off — higher cost in exchange for the best convergence accuracy — must be evaluated in the context of the target application's constraints.

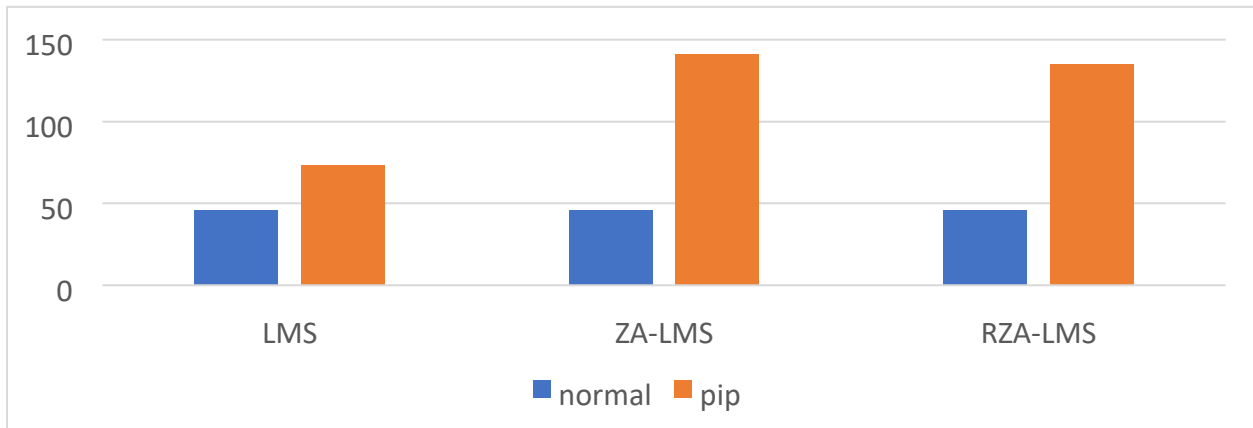


Fig 21. Fmax comparison

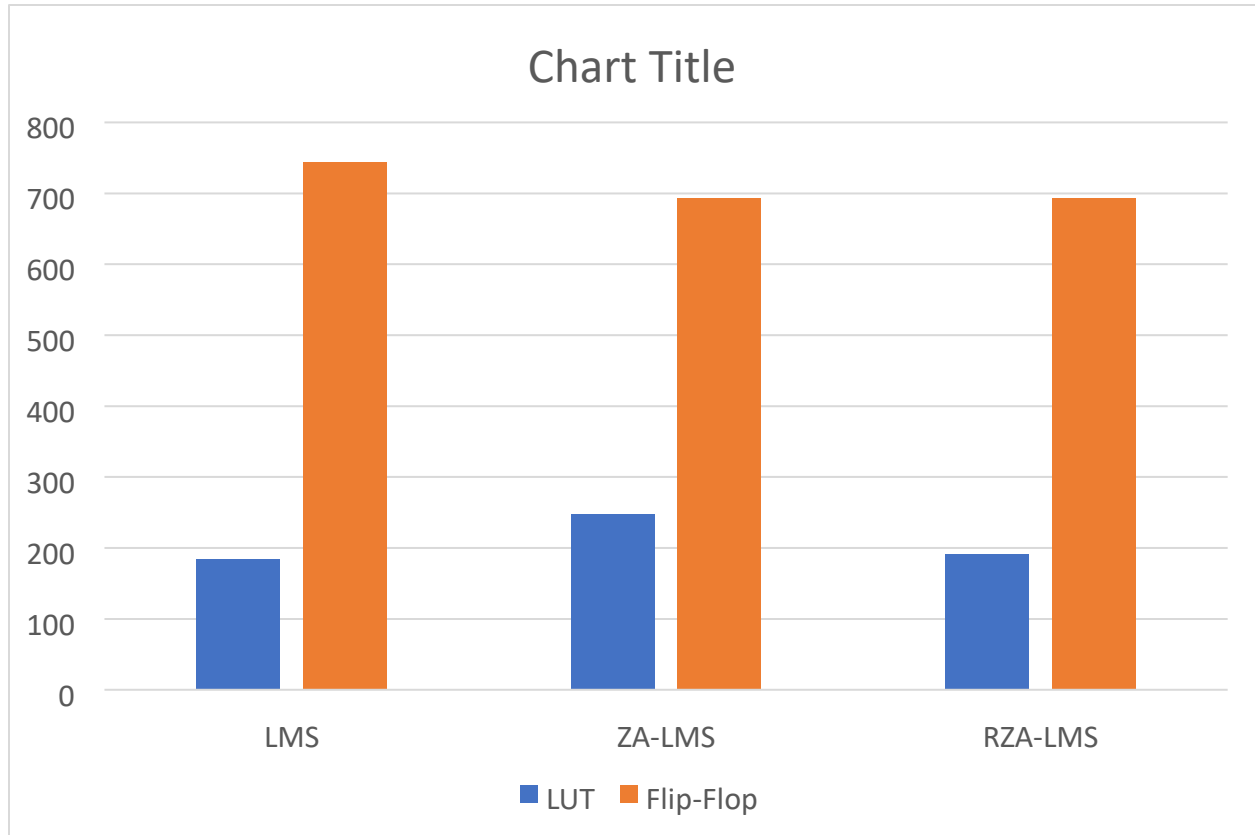


Fig 22. LUT and flip-flop comparison

7.4 Summary and Discussion

The comparative analysis presented in this chapter reveals a consistent and interpretable set of trade-offs across the four adaptive filter implementations, each occupying a distinct position in the **convergence–speed–complexity** design space.

Standard **LMS** is the simplest implementation in terms of both algorithm and hardware, but it is the weakest performer on sparse channels and the most constrained in clock frequency. It remains the appropriate choice only when the unknown system is known to be dense and hardware throughput is not a critical concern.

DLMS does not improve convergence quality — its steady-state MSE is comparable to LMS and its convergence speed is slower. Its value is entirely in the hardware domain: the pipelined architecture achieves the highest Fmax of all four designs, making it the preferred choice when maximum throughput is the primary objective and the application can tolerate slower adaptation.

ZA-LMS is the most practically balanced implementation for sparse system identification. Its hardware overhead over LMS is small, its convergence improvement is significant, and its implementation complexity is manageable. For applications where the channel is known to be sparse and hardware resources are limited, ZA-LMS offers the best cost-performance ratio among the sparsity-aware algorithms.

RZA-LMS delivers the best algorithmic performance — fastest convergence and lowest steadystate MSE on sparse channels — at the cost of the highest hardware complexity and resource utilization. Its suitability depends on whether the application's performance requirements justify the additional LUT, DSP, and timing overhead relative to ZA-LMS.

Taken together, these results validate the central thesis of this work: that the LMS algorithm family can be systematically extended — through pipelining for hardware efficiency and through sparsityaware updates for algorithmic performance — with each extension offering a well-defined and measurable trade-off that can be rationally evaluated for a given application context.

CHAPTER 8

CONCLUSIONS AND FUTURE SCOPE

8.1 Summary of Contributions

This work presented the design, implementation, simulation, and synthesis of four adaptive filter variants **LMS**, **DLMS**, **ZA-LMS**, and **RZA-LMS** targeting the problem of sparse system identification on an FPGA platform. All implementations were developed in synthesizable Verilog HDL using **Q4.12 signed fixed-point arithmetic**, simulated using Python-generated test stimuli, and evaluated through Vivado post-synthesis reports. The work spans both the algorithmic and hardware dimensions of adaptive filtering, providing a unified treatment of the speed-convergence-sparsity trade-off space.

The **baseline LMS implementation** established the fundamental adaptive filter architecture a 8-tap FIR core coupled with a gradient-based weight update — and characterized its convergence behavior and synthesis performance. The critical path analysis of LMS identified the weight-feedback dependency as the primary bottleneck limiting clock frequency, motivating the pipelined extension.

The **DLMS implementation** with pipeline depth $L = 8$ demonstrated that the LMS feedback loop can be broken through retiming, yielding a measurable improvement in F_{max} at the cost of a reduced step-size bound and slower iteration-wise convergence. The weight delay and error-input delay buffers were implemented in Verilog and their correctness verified through simulation, confirming that DLMS converges to the same Wiener solution as LMS under stationary conditions.

The **ZA-LMS implementation** introduced the Zero Attractor Unit into the weight update datapath, enabling sparsity-aware adaptation through an L1 norm penalty. Simulation on a sparse 16-tap channel demonstrated improved convergence speed and lower steady-state MSE compared to standard LMS, with only a modest increase in hardware resource utilization. ZA-LMS emerged as the most practically balanced design for sparse channel applications.

The **RZA-LMS implementation** extended ZA-LMS with a reweighted attraction mechanism, eliminating the dominant-tap bias inherent in uniform zero-attraction. The fixed-point

reciprocal approximation required by the Reweighted Zero Attractor Unit was addressed through a LUT-based approach, and its precision trade-offs were analyzed. RZA-LMS achieved the best convergence performance among all four algorithms, at the cost of the highest hardware complexity.

8.2 Limitations of the Current Work

Several practical constraints bound the scope of the results presented. First, all implementations were evaluated through **post-synthesis simulation** in Vivado and were not deployed on physical FPGA hardware. While synthesis reports provide reliable estimates of timing and resource utilization, on-board testing would validate the design under real operating conditions including clock distribution, I/O constraints, and power supply noise.

Second, the **reciprocal approximation** in RZA-LMS introduces a quantization error whose impact on convergence was observed qualitatively but not rigorously bounded analytically. A formal fixed-point error analysis, characterizing the relationship between LUT table size, approximation error, and steady-state MSE degradation, remains an open task.

Third, all simulations used **stationary, synthetically generated** sparse channels. Realworld sparse channels are often **time-varying** — the sparsity pattern and dominant tap locations change over time — and the tracking behavior of ZA-LMS and RZA-LMS under non-stationary conditions was not evaluated in this work.

8.3 Directions for Future Work

Several natural extensions follow directly from the contributions of this work.

The most immediate extension is the development of a **pipelined ZA-LMS (ZA-DLMS)** architecture, combining the $L = 8$ pipeline structure of DLMS with the zero-attraction mechanism of ZA-LMS. Such a design would target both high throughput and sparse channel performance simultaneously — a combination not addressed by any single implementation in this work. The primary challenge is correctly aligning the delayed weight vector and zero-attraction correction within the pipelined update equation.

A second direction is the implementation of **post-implementation timing analysis and on-board FPGA verification**. Deploying the designs on a physical Xilinx FPGA — using external

signal generation for the input and desired signals, and capturing weight convergence through onchip logic analyzers such as Vivado's **Integrated Logic Analyzer (ILA)** — would provide empirical validation of the synthesis-based performance claims.

Third, extending the filter length beyond $N = 8$ to **$N = 64$ or $N = 128$** would test the scalability of the architectures, particularly the resource efficiency of the parallel weight update structure and the LUT-based reciprocal approximation in RZA-LMS, which grows linearly with filter order.

Finally, exploring **adaptive selection of the sparsity parameter ρ** — adjusting it dynamically based on the estimated sparsity of the current channel — represents a promising algorithmic direction. Hardware realization of such an adaptive parameter scheme would require additional logic for sparsity estimation but could yield significantly improved robustness across channels of varying sparsity levels

REFERENCES

- [1] B. K. Das and M. Chakraborty, "Sparse adaptive filtering by an adaptive convex combination of the LMS and the ZA-LMS algorithms," *IEEE Transactions on Circuits and Systems I: Regular Papers*, vol. 61, no. 5, pp. 1499–1507, May 2014.
<https://ieeexplore.ieee.org/document/6704331>
- [2] Y. Chen, Y. Gu, and A. O. Hero, "Sparse LMS for system identification," in *Proc. IEEE ICASSP*, Taipei, Taiwan, Apr. 2009, pp. 3125–3128.
<https://ieeexplore.ieee.org/document/4960286>
- [3] G. Gui, L. Xu, and F. Adachi, "Improved adaptive sparse channel estimation," *Wireless Communications and Mobile Computing*, 2014.
<https://onlinelibrary.wiley.com/doi/full/10.1002/wcm.2453>
- [4] Y. Chen, Y. Gu, and A. O. Hero, "Sparse LMS for system identification," *Signal Processing*, 2010. <https://www.sciencedirect.com/science/article/pii/S0165168410002173>
- [5] L. Luo and W.-Z. Zhu, "An optimized zero-attracting LMS algorithm for the identification of sparse system," *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, vol. 30, pp. 3060–3073, 2022, doi: 10.1109/TASLP.2022.3209946.
<https://ieeexplore.ieee.org/abstract/document/9903570>
- [6] W. Qin et al., "Sparse LMS with segment zero attractors," in *Proc. IEEE ICASSP*, 2011.
<https://ieeexplore.ieee.org/abstract/document/5774742>
- [7] D. Jin, J. Chen, C. Richard, and J. Chen, "Model-driven online parameter adjustment for zero-attracting LMS," *Signal Processing*, vol. 152, pp. 373–383, 2018, doi: 10.1016/j.sigpro.2018.06.020. <https://www.sciencedirect.com/science/article/pii/S0165168418302172>
- [8] J. Chen, C. Richard, Y. Song, and D. Brie, "Transient performance analysis of zero-attracting LMS," *IEEE Signal Processing Letters*, vol. 23, no. 12, pp. 1786–1790, Dec. 2016, doi: 10.1109/LSP.2016.2616890. <https://ieeexplore.ieee.org/abstract/document/7589076>
- [9] Y. Guo, H. Wang, and L. Li, "Improved zero-attracting LMS algorithm for the adaptive identification of sparse system," in *Proc. IEEE 5th International Conference on Electronic*

Information and Communication Technology (ICEICT), Hefei, China, 2022, pp. 196–201, doi: 10.1109/ICEICT55736.2022.9909029. <https://ieeexplore.ieee.org/abstract/document/9909029>

[10] Y. Wei, Z. Wang, and Y. Zhang, "A reweighted zero-attracting/repelling LMS algorithm for sparse system identification," in *Proc. 2nd International Conference on Image, Vision and Computing (ICIVC)*, Chengdu, China, 2017, pp. 1128–1132, doi: 10.1109/ICIVC.2017.7984729. <https://ieeexplore.ieee.org/abstract/document/7984729>

[11] S. C. Douglas, "A pipelined LMS adaptive FIR filter architecture without adaptation delay," *IEEE Transactions on Signal Processing*, vol. 46, no. 3, pp. 775–779, 1998. <https://ieeexplore.ieee.org/abstract/document/661345>

[12] L.-K. Ting, R. Woods, and C. F. N. Cowan, "Virtex FPGA implementation of a pipelined adaptive LMS predictor for electronic support measures receivers," *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, vol. 13, no. 1, pp. 86–95, Jan. 2005, doi: 10.1109/TVLSI.2004.840403. <https://ieeexplore.ieee.org/abstract/document/1386267>

[13] M. T. Khan and R. A. Shaik, "Optimal complexity architectures for pipelined distributed arithmetic-based LMS adaptive filter," *IEEE Transactions on Circuits and Systems I: Regular Papers*, vol. 66, no. 2, pp. 630–642, Feb. 2019, doi: 10.1109/TCSI.2018.2867291. <https://ieeexplore.ieee.org/abstract/document/8466615>

[14] N. R. Shanbhag and K. K. Parhi, "A pipelined LMS adaptive filter architecture," in *Proc. IEEE Asilomar Conference on Signals, Systems and Computers*, 1991. <https://ieeexplore.ieee.org/abstract/document/112086>

[15] L.-D. Van and W.-S. Feng, "An efficient systolic architecture for the DLMS adaptive filter and its applications," *IEEE Transactions on Circuits and Systems II: Analog and Digital Signal Processing*, vol. 48, no. 4, pp. 359–366, Apr. 2001, doi: 10.1109/82.933794. <https://ieeexplore.ieee.org/abstract/document/933794>

[16] P. S. R. Diniz, "The least-mean-square (LMS) algorithm," in *Adaptive Filtering*. Cham: Springer, 2020, doi: 10.1007/978-3-030-29057-3_3. https://link.springer.com/chapter/10.1007/978-3-030-29057-3_3

[17] M. D. Meyer and D. P. Agrawal, "A high sampling rate delayed LMS filter architecture," *IEEE Transactions on Circuits and Systems II: Analog and Digital Signal Processing*, vol.

- 40, no. 11, pp. 727–729, Nov. 1993, doi: 10.1109/82.251841.
<https://ieeexplore.ieee.org/abstract/document/251841>
- [18] Y. Yi, R. Woods, L.-K. Ting, and C. F. N. Cowan, "High speed FPGA-based implementations of delayed-LMS filters," *Journal of VLSI Signal Processing*, vol. 39, pp. 113–131, 2005, doi: 10.1023/B:VLSI.00000047275.54691.be.
<https://link.springer.com/article/10.1023/B:VLSI.00000047275.54691.be>
- [19] S. Dixit and D. Nagaria, "LMS adaptive filters for noise cancellation: A review," *International Journal of Electrical and Computer Engineering*, Amity University, Noida and Bundelkhand Institute of Engineering and Technology, Jhansi.
<https://pdfs.semanticscholar.org/9f62/60e7f629965ddd12c1933d70765a4d93f4e1.pdf>
- [20] N. R. Shanbhag and K. K. Parhi, "A pipelined LMS adaptive filter architecture," in *Proc. 25th Asilomar Conference on Signals, Systems and Computers*, Pacific Grove, CA, USA, 1991, vol. 2, pp. 668–672, doi: 10.1109/ACSSC.1991.186532.
<https://ieeexplore.ieee.org/document/186532>
- [21] S. Ramanathan and V. Visvanathan, "Low-power pipelined LMS adaptive filter architectures with minimal adaptation delay," *Integration, the VLSI Journal*, vol. 27, no. 1, pp. 1–32, 1999, doi: 10.1016/S0167-9260(98)00013-3.
<https://www.sciencedirect.com/science/article/pii/S0167926098000133>